



UNIVERSIDAD MAYOR DE
SAN SIMÓN

FACULTAD DE CIENCIAS Y
TECNOLOGÍA

CARRERA DE INFORMÁTICA



Ingeniería Informática

INFORME FINAL

Gestión de Comandas

Sistema móvil para la gestión de pedidos, ventas,
gastos y reportes financieros en restaurantes

Materia: Programación Móvil

Docente: Dr. Américo Fiorilo Lozada Ph.D.

INTEGRANTES:

Nombre	Carrera	Rol
Acebey Zalazar Jimmy Randall	Ing. Informática	Dev
Cáceres Véliz Erick Alvaro	Ing. Informática	Dev
Revollo Bernal Amilcar Diego	Ing. Informática	SM
Siles Espicho Adrian Eloy	Ing. Informática	Dev
Tinta Mamani Jairo Amado	Ing. Informática	Dev

Carrera: Ing. Informática

Fecha: 05/06/2026

COCHABAMBA — BOLIVIA

Índice general

1. Resumen Ejecutivo	4
1.1. Problemática Resuelta	4
1.2. Solución Implementada	4
1.3. Metodología y Gestión	5
2. Análisis del Proyecto	6
2.1. Objetivos	6
2.1.1. Objetivo General	6
2.1.2. Objetivos Específicos	6
2.2. Problema o Necesidad Identificada	6
2.3. Contexto	7
2.4. Requisitos Funcionales	7
2.5. Requisitos No Funcionales	8
2.5.1. Historias de Usuario	8
2.5.2. Setup e infraestructura	9
2.5.3. Autenticación y roles	13
2.5.4. Row Level Security	17
2.5.5. Gestión de usuarios (admin)	19
2.5.6. Gestión de mesas (admin)	22
2.5.7. Gestión de menú (admin)	24
2.5.8. Toma de pedidos (mesero)	29
2.5.9. Pagos (admin)	36
2.5.10. Gastos operativos (admin)	39
2.5.11. Vista de mesas en tiempo real (mesero)	41
2.5.12. Llamada al mesero (cliente → mesero)	45
2.5.13. Carta digital pública (web)	48
2.5.14. Sesión de cliente en mesa (web)	51
2.5.15. Reportes financieros (admin)	54

3. Diseño del Proyecto	58
3.1. Stack Tecnológico	58
3.2. Modelo de Datos	59
3.2.1. Entidades y Relaciones	59
3.2.2. Esquema de Tablas	60
3.3. Estructura del Proyecto	62
3.4. Roles y Accesos	63
4. Arquitectura del Sistema	64
4.1. Diagrama de Contexto	64
4.2. Diagrama de Contenedores	65
4.3. Diagrama de Componentes — Funcionalidades Clave	66
4.3.1. Componentes de la aplicación móvil	66
4.3.2. Componentes de rutas web públicas	66
4.3.3. Contexto de Supabase	67
4.3.4. Contenedores de Supabase	68
4.3.5. Componentes de la Base de Datos	68
4.3.6. Vista de Estado de Mesas (Mesero)	69
5. Metodología Aplicada	70
5.1. Metodología de Desarrollo	70
5.2. Ciclo de Vida del Proyecto	70
5.3. Herramientas de Gestión	70
6. Seguimiento y Desarrollo Grupal	72
6.1. Planificación del Trabajo	72
6.2. Roles y Responsabilidades	74
6.3. Control de Versiones	75
6.4. Reporte de Avances	75
6.4.1. Indicadores de Progreso	75
6.4.2. Dificultades Encontradas	77
6.5. Comunicación	77

7. Pruebas y Resultados	78
7.1. Plan de Pruebas	78
7.1.1. Tipos de Pruebas Aplicadas	78
7.1.2. Escenarios Evaluados	78
7.1.3. Herramientas de Soporte	79
7.1.4. Condiciones de Aprobación	79
7.2. Resultados Obtenidos	79
7.2.1. Registro y Gestión de Pedidos	79
7.2.2. Vista de Mesas en Tiempo Real	80
7.2.3. Carta Digital y Sesión de Cliente	80
7.2.4. Reportes Financieros	80
7.2.5. Seguridad RLS	80
7.3. Limitaciones Conocidas	80
7.3.1. Free Tier de Supabase	80
7.3.2. Limitaciones de Alcance Documentadas	80
7.4. Mejoras Propuestas	81
8. Conclusiones	82
8.1. Recomendaciones	82
8.1.1. Futuros Desarrollos	82
8.1.2. Optimizaciones Técnicas	83
8.1.3. Gestión y Comunicación	83
9. Bibliografía	84
9.1. Tecnologías y Frameworks	84
9.2. Herramientas de Desarrollo	84
9.3. Referencias Institucionales	84

1. Resumen Ejecutivo

El proyecto **Gestión de Comandas** consiste en el desarrollo de una aplicación móvil integral diseñada para centralizar y gestionar operativamente los pedidos, ventas, gastos y reportes financieros de restaurantes pequeños y medianos. La plataforma aborda la problemática del registro manual en papel mediante una solución tecnológica que permite a meseros y administradores operar en tiempo real desde dispositivos Android, complementada con una carta digital pública y un sistema de interacción directa del cliente desde su mesa.

1.1 Problemática Resuelta

La aplicación resuelve los principales desafíos identificados en la operación diaria de restaurantes:

- **Errores de cálculo** en el cierre diario de caja por registros manuales.
- **Dificultad de conciliación** entre ventas registradas e ingresos reales.
- **Ausencia de visibilidad financiera** en tiempo real para el administrador.
- **Procesos lentos** de verificación al final de cada turno.
- **Falta de trazabilidad** del estado de cada mesa durante el servicio.
- **Ausencia de un canal directo** entre cliente y mesero sin interrupciones físicas.

1.2 Solución Implementada

Se desarrolló una aplicación móvil nativa para Android con las siguientes características principales:

Arquitectura Técnica:

- *Frontend móvil*: React Native con Expo SDK 52+
- *Web (carta digital + sesión de cliente)*: Expo Router (rutas públicas web)
- *BaaS*: Supabase (PostgreSQL + Auth + Storage + Realtime)
- *Lenguaje*: TypeScript
- *Deploy*: Expo EAS Build + EAS Update (OTA)

Funcionalidades Clave:

- Registro de pedidos con selección de platillos, mesa y método de pago.
- Vista en tiempo real del estado de mesas para el mesero.

- Notificaciones push cuando un pedido pasa a estado *listo*.
- Carta digital pública accesible por QR sin necesidad de app ni registro.
- Sesión anónima del cliente por mesa: ver menú y llamar al mesero.
- Gestión completa del menú (CRUD + imágenes vía Supabase Storage).
- Registro de gastos operativos y generación de reportes financieros.
- Acceso diferenciado por rol: `waiter`, `admin`, `customer`.

1.3 Metodología y Gestión

El proyecto se desarrolló bajo la metodología ágil **Scrum**, organizado en **3 sprints de 4 semanas** cada uno, con un equipo multidisciplinario de 5 integrantes que cumplieron roles específicos de desarrollo y gestión. Se utilizaron herramientas modernas de colaboración como GitHub, Notion, Discord y Excalidraw para garantizar una comunicación efectiva y un control de versiones robusto.

2. Análisis del Proyecto

2.1 Objetivos

2.1.1 Objetivo General

Desarrollar una aplicación móvil que permita registrar ventas, gestionar pagos, controlar gastos, generar reportes financieros y visualizar el estado del servicio en tiempo real en restaurantes pequeños y medianos.

2.1.2 Objetivos Específicos

1. Permitir a los meseros registrar pedidos y pagos (efectivo o QR) desde dispositivos móviles.
2. Implementar un sistema de gestión de menú con actualización de precios y disponibilidad en tiempo real.
3. Registrar gastos operativos del restaurante (insumos, proveedores, etc.).
4. Generar reportes automáticos de ingresos brutos, netos, egresos y ranking de platos más vendidos.
5. Implementar acceso diferenciado por rol: `waiter`, `admin` y `customer`.
6. Notificar a los meseros en tiempo real cuando un pedido está listo para ser entregado.
7. Proveer al mesero una vista en tiempo real del estado de todas sus mesas.
8. Exponer una carta digital pública accesible por QR para los clientes del restaurante.
9. Permitir al cliente ver el menú completo con disponibilidad real y llamar al mesero desde su dispositivo, sin descargar ninguna app.

2.2 Problema o Necesidad Identificada

Los restaurantes pequeños y medianos continúan registrando pedidos en papel, lo que genera una serie de problemas operacionales recurrentes:

1. **Errores de cálculo en caja:** El cierre diario depende de sumas manuales que son propensas a errores humanos.
2. **Falta de conciliación:** Es difícil comparar ventas registradas con ingresos reales, especialmente cuando se mezclan pagos en efectivo y QR.
3. **Sin visibilidad financiera en tiempo real:** El administrador no cuenta con datos inmediatos sobre el desempeño del día.
4. **Procesos lentos de verificación:** Al final de cada turno, validar cada comanda consume tiempo significativo del personal.

5. **Sin trazabilidad de mesas:** No existe un mecanismo para conocer el estado de cada mesa (libre, ocupada, con pedido listo) sin acercarse físicamente.
6. **Sin canal cliente–mesero:** El cliente debe interrumpir al mesero para solicitar atención, lo que genera fricciones durante el servicio.

2.3 Contexto

El sistema está diseñado para restaurantes de tamaño pequeño a mediano que buscan modernizar su gestión operativa sin inversión en infraestructura de servidores propios. El modelo BaaS (Backend as a Service) sobre Supabase permite desplegar toda la solución sin administrar servidores, escalando automáticamente con la demanda.

El público objetivo comprende:

- **Meseros:** usuarios principales del día a día, con necesidad de interfaces rápidas e intuitivas.
- **Administradores:** responsables de la gestión del menú, gastos y reportes financieros.
- **Cientes en mesa:** usuarios ocasionales que interactúan mediante sesión anónima vía QR, sin necesidad de registro ni descarga de app.

2.4 Requisitos Funcionales

ID	Requisito
RF-01	Registro de pedidos con selección de platillos, mesa y método de pago.
RF-02	Cálculo automático del total por venta.
RF-03	Registro de gastos diarios por parte del administrador.
RF-04	Actualización del menú (precios, disponibilidad, imágenes) en tiempo real.
RF-05	Generación de reportes de ingresos brutos, netos y egresos (diario, semanal, mensual).
RF-06	Reporte de platos más vendidos con ranking por período.
RF-07	Acceso diferenciado: mesero solo accede a pedidos y mesas; admin accede a todo.
RF-08	Notificaciones en tiempo real al mesero cuando el pedido pasa a estado ready .
RF-09	Vista de estado de mesas para el mesero: pendiente, listo, entregado, libre.
RF-10	Carta digital pública accesible mediante QR, sin necesidad de app ni registro.
RF-11	El cliente puede escanear un QR por mesa para iniciar sesión anónima y ver el menú con disponibilidad real.
RF-12	El cliente puede llamar al mesero desde su dispositivo, generando una notificación en tiempo real al mesero asignado.

ID	Requisito
----	-----------

Cuadro 2.1. *Requisitos Funcionales del Sistema*

2.5 Requisitos No Funcionales

ID	Requisito
RNF-01	Interfaz intuitiva con curva de aprendizaje mínima.
RNF-02	Cifrado de datos sensibles (transacciones, sesiones).
RNF-03	Escalable para múltiples sucursales.
RNF-04	Disponibilidad alta con recuperación ante fallos.
RNF-05	Compatible con Android.
RNF-06	Procesamiento de pedidos sin latencia perceptible en horas pico.
RNF-07	Actualizaciones sin interrupciones en el servicio (OTA vía Expo EAS Update).
RNF-08	Carta digital accesible desde cualquier dispositivo con navegador web.
RNF-09	La sesión del cliente en mesa no requiere registro ni descarga de aplicación.

Cuadro 2.2. *Requisitos No Funcionales del Sistema*

2.5.1 Historias de Usuario

Las historias de usuario representan los requerimientos funcionales del sistema desde la perspectiva de los distintos actores involucrados, principalmente administradores, meseros y clientes.

Cada historia describe una necesidad específica del usuario junto con el valor que aporta al sistema, permitiendo organizar el desarrollo mediante iteraciones incrementales.

2.5.2 Setup e infraestructura

HU-01 — Inicializar proyecto Expo con TypeScript y Expo Router

HU-01	Inicializar proyecto Expo con TypeScript y Expo Router		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como desarrollador, quiero tener el proyecto base configurado con TypeScript y Expo Router para comenzar a construir sobre una estructura sólida y predecible desde el primer sprint.			
Criterios de aceptación 1. El proyecto compila y corre en Android sin errores ni advertencias críticas. 2. Expo Router está configurado como sistema de navegación principal con soporte para grupos de rutas. 3. <code>tsconfig.json</code> tiene el modo estricto activado (<code>"strict": true</code>). 4. La estructura de carpetas sigue la convención <code>app/</code> con grupos (<code>auth</code>), (<code>waiter</code>) y (<code>admin</code>). 5. Las variables de entorno están definidas en <code>.env</code> y excluidas del repositorio vía <code>.gitignore</code>. 6. El proyecto tiene un <code>README.md</code> con instrucciones mínimas de instalación y ejecución local. 7. El repositorio está inicializado en Git con una rama <code>main</code> protegida.			

Cuadro 2.3. *HU-01: Configuración inicial del proyecto*

HU-02 — Configurar proyecto Supabase y variables de entorno

HU-02	Configurar proyecto Supabase y variables de entorno		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como desarrollador, quiero tener el cliente Supabase conectado al proyecto para poder consumir todos sus servicios desde la aplicación móvil.			
Criterios de aceptación 1. <code>supabase-js</code> v2 está instalado como dependencia del proyecto. 2. <code>lib/supabase.ts</code> expone el cliente autenticado con la <code>SUPABASE_URL</code> y <code>SUPABASE_ANON_KEY</code>. 3. <code>lib/supabase-public.ts</code> expone un cliente separado para uso anónimo en carta digital y sesión de cliente. 4. El archivo <code>.env</code> está incluido en <code>.gitignore</code> y se provee un <code>.env.example</code> con las claves vacías.			

Cuadro 2.4. *HU-02: Configuración del cliente Supabase***HU-03 — Crear migración de esquema inicial de la base de datos**

HU-03	Crear migración de esquema inicial de la base de datos		
Estado	Sin empezar	Asignado	—
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como desarrollador, quiero tener todas las tablas del sistema creadas mediante migraciones versionadas para mantener el historial del esquema y poder replicar el entorno en cualquier momento.			
Criterios de aceptación 1. La migración <code>001_schema.sql</code> crea las tablas: <code>restaurants</code>, <code>tables</code>, <code>users</code>, <code>menu_items</code>, <code>orders</code>, <code>order_items</code>, <code>payments</code>, <code>expenses</code> y <code>table_calls</code>. 2. Todos los enums del sistema están definidos: <code>order_status</code>, <code>payment_method</code>, <code>payment_status</code> y <code>user_role</code>. 3. La migración se ejecuta sin errores en un proyecto Supabase limpio. 4. El tipo de dato <code>decimal(10,2)</code> se usa para todos los campos monetarios.			

Cuadro 2.5. *HU-03: Migración del esquema inicial*

HU-04 — Crear constraint de orden activa única por mesa

HU-04		Crear constraint de orden activa única por mesa	
Estado	Sin empezar	Asignado	—
Esfuerzo	1 punto	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero evitar que existan dos órdenes activas en la misma mesa simultáneamente para mantener la integridad del servicio y prevenir conflictos de estado.			
Criterios de aceptación 1. El índice único parcial <code>one_active_order_per_table</code> está creado sobre <code>orders(table_id)</code>. 2. El índice excluye órdenes con <code>status</code> IN (<code>'delivered'</code>, <code>'cancelled'</code>). 3. Intentar insertar una segunda orden activa en la misma mesa falla con error de constraint a nivel de base de datos. 4. Una vez que la orden pasa a <code>delivered</code> o <code>cancelled</code>, la misma mesa acepta una nueva orden sin errores. 5. El constraint está documentado en la migración correspondiente con un comentario explicativo. 6. El comportamiento es verificable mediante pruebas manuales con datos de prueba.			

Cuadro 2.6. *HU-04: Constraint de orden activa única por mesa*

HU-05 — Crear trigger set_closed_at al entregar una orden

HU-05		Crear trigger set_closed_at al entregar una orden	
Estado	Sin empezar	Asignado	—
Esfuerzo	1 punto	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero que <code>closed_at</code> se asigne automáticamente al pasar una orden a estado <code>delivered</code> para no depender del cliente y garantizar la integridad del historial financiero.			
Criterios de aceptación 1. La función <code>set_closed_at()</code> y el trigger <code>trg_set_closed_at</code> están creados en la migración <code>004_triggers.sql</code>. 2. Al ejecutar <code>UPDATE orders SET status='delivered'</code>, el campo <code>closed_at</code> se puebla automáticamente con <code>now()</code>. 3. El trigger es de tipo <code>BEFORE UPDATE</code> para que el valor quede disponible en la misma transacción. 4. Si la orden ya tenía <code>status='delivered'</code>, una actualización posterior no sobrescribe <code>closed_at</code>. 5. Las órdenes con <code>status='cancelled'</code> no reciben valor en <code>closed_at</code>. 6. El comportamiento es verificable ejecutando un <code>UPDATE</code> directo en la base de datos y consultando el campo resultante.			

Cuadro 2.7. *HU-05: Trigger de cierre automático de orden*

2.5.3 Autenticación y roles

HU-06 — Implementar pantalla de login para mesero y admin

HU-06	Implementar pantalla de login para mesero y admin		
Estado	Sin empezar	Asignado	—
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero o administrador, quiero poder iniciar sesión con mi email y contraseña para acceder al sistema según el rol que me corresponde.			
Criterios de aceptación 1. La pantalla presenta un formulario con campos de email y contraseña. 2. Al autenticarse con credenciales correctas, el sistema redirige a (waiter) o (admin) según el rol en user_metadata. 3. Si las credenciales son incorrectas, se muestra un mensaje de error visible sin exponer detalles técnicos. 4. El campo de contraseña enmascara el texto por defecto, con opción de mostrarlo. 5. La sesión se persiste localmente para que el usuario no deba iniciar sesión en cada apertura de la app. 6. El botón de envío está deshabilitado mientras se procesa la petición para evitar envíos duplicados. 7. El formulario valida que los campos no estén vacíos antes de llamar a Supabase Auth.			

Cuadro 2.8. HU-06: Pantalla de inicio de sesión

HU-07 — Proteger rutas según rol con guards de navegación

HU-07	Proteger rutas según rol con guards de navegación		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero que un mesero no pueda acceder a las rutas del administrador y viceversa para respetar el acceso diferenciado definido en los requisitos funcionales.			
Criterios de aceptación 1. Un usuario con <code>role=waiter</code> que intenta acceder a rutas (<code>admin</code>) es redirigido a la pantalla de login. 2. Un usuario con <code>role=admin</code> que intenta acceder a rutas (<code>waiter</code>) es redirigido al área de admin. 3. Un usuario no autenticado que intenta acceder a cualquier ruta protegida es redirigido a la pantalla de login. 4. La sesión se recupera del almacenamiento local al reiniciar la app sin requerir nuevo login. 5. El guard lee el rol desde <code>user_metadata</code> del JWT, no desde una llamada adicional a la base de datos. 6. Los guards están implementados en los <code>_layout.tsx</code> de cada grupo de rutas.			

Cuadro 2.9. HU-07: Guards de navegación por rol

HU-08 — Crear trigger sync_role_to_auth para sincronizar rol

HU-08	Crear trigger sync_role_to_auth para sincronizar rol		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero que el rol de la tabla <code>users</code> se sincronice automáticamente hacia <code>user_metadata</code> de Supabase Auth para que los JWT siempre reflejen el rol actualizado sin intervención manual.			
Criterios de aceptación 1. El trigger <code>trg_sync_role</code> es de tipo <code>AFTER UPDATE OF role</code> sobre la tabla <code>users</code> . 2. Al modificar el campo <code>role</code> en <code>users</code> , el campo <code>raw_user_meta_data</code> en <code>auth.users</code> se actualiza automáticamente. 3. La función del trigger usa <code>SECURITY DEFINER</code> para poder escribir en el esquema <code>auth</code> . 4. El nuevo rol queda disponible en el JWT del usuario en la siguiente sesión iniciada. 5. La sincronización no afecta otros campos de <code>raw_user_meta_data</code> existentes. 6. El trigger está definido en la migración <code>004_triggers.sql</code> y versionado en el repositorio.			

Cuadro 2.10. *HU-08: Trigger de sincronización de rol a Supabase Auth*

HU-09 — Implementar cierre de sesión

HU-09	Implementar cierre de sesión		
Estado	Sin empezar	Asignado	—
Esfuerzo	1 punto	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero o administrador, quiero poder cerrar mi sesión desde la aplicación para que nadie más pueda operar el sistema desde mi dispositivo.			
Criterios de aceptación 1. El botón de cierre de sesión está accesible desde la navegación principal de cada rol. 2. Al cerrar sesión, se invoca <code>supabase.auth.signOut()</code> y la sesión local queda destruida. 3. Tras el cierre, el usuario es redirigido a la pantalla de login. 4. Intentar navegar hacia atrás tras cerrar sesión no permite acceder a rutas protegidas. 5. El botón de logout muestra un estado de carga mientras se procesa la petición. 6. La acción no genera errores si la sesión ya había expirado previamente.			

Cuadro 2.11. *HU-09: Cierre de sesión*

2.5.4 Row Level Security

HU-10 — Implementar RLS para la tabla orders

HU-10	Implementar RLS para la tabla orders		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero que un mesero solo pueda ver y modificar sus propias órdenes, y que el administrador tenga acceso total a todas las órdenes del restaurante.			
Criterios de aceptación 1. RLS está habilitado en la tabla <code>orders</code>. 2. La política <code>waiter_own_orders</code> restringe <code>SELECT</code> al <code>waiter_id = auth.uid()</code>. 3. La política <code>waiter_insert_orders</code> permite <code>INSERT</code> solo si el rol en el JWT es <code>waiter</code>. 4. La política <code>waiter_update_own_orders</code> permite <code>UPDATE</code> solo sobre las órdenes propias del mesero. 5. La política <code>admin_all_orders</code> concede acceso <code>ALL</code> a usuarios con rol <code>admin</code>. 6. Un mesero que intenta leer órdenes de otro mesero recibe un resultado vacío, no un error. 7. Las políticas están definidas en la migración <code>002_rls.sql</code>.			

Cuadro 2.12. HU-10: RLS en la tabla orders

HU-11 — Implementar RLS para expenses, menu_items y table_calls

HU-11	Implementar RLS para expenses, menu_items y table_calls		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero que los gastos solo los pueda insertar el administrador, que el menú sea legible públicamente con las restricciones correctas y que las llamadas al mesero estén controladas por rol.			
Criterios de aceptación 1. La política <code>admin_insert_expenses</code> restringe INSERT en <code>expenses</code> a rol <code>admin</code>. 2. La política <code>public_menu_read</code> permite SELECT en <code>menu_items</code> únicamente sobre ítems con <code>available=true</code> sin autenticación. 3. La política <code>customer_full_menu_read</code> permite SELECT del menú completo a usuarios con rol <code>customer</code>. 4. La política <code>customer_insert_table_call</code> permite INSERT en <code>table_calls</code> solo si el <code>table_id</code> del JWT coincide con el de la fila. 5. La política <code>waiter_table_calls</code> permite al mesero ver y actualizar llamadas de las mesas con órdenes activas que le pertenecen. 6. Un cliente anónimo sin sesión no puede insertar en <code>table_calls</code>. 7. Todas las políticas están definidas en la migración <code>002_rls.sql</code>.			

Cuadro 2.13. HU-11: RLS en expenses, menu_items y table_calls

2.5.5 Gestión de usuarios (admin)

HU-12 — Listar usuarios del sistema

HU-12	Listar usuarios del sistema		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero ver la lista de meseros y admins registrados para conocer el equipo activo del restaurante y gestionar sus accesos.			
Criterios de aceptación 1. La pantalla muestra nombre, email, rol y estado activo de cada usuario. 2. Solo usuarios con rol admin pueden acceder a esta pantalla. 3. La lista diferencia visualmente entre roles waiter y admin. 4. Los usuarios inactivos aparecen en la lista pero con indicador visual de su estado. 5. La pantalla no expone el hash de contraseña ni datos sensibles de autenticación. 6. Los datos se obtienen desde la tabla users, no directamente desde auth.users.			

Cuadro 2.14. *HU-12: Listado de usuarios del sistema*

HU-13 — Crear nuevo usuario mesero o admin

HU-13	Crear nuevo usuario mesero o admin		
Estado	Sin empezar	Asignado	—
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero poder crear cuentas para nuevos integrantes del equipo para que puedan acceder al sistema con el rol correspondiente desde el primer día.			
Criterios de aceptación 1. El formulario incluye campos de nombre, email, rol y contraseña temporal. 2. El usuario se crea en Supabase Auth y simultáneamente en la tabla <code>users</code> con el mismo id. 3. El rol queda sincronizado en <code>user_metadata</code> mediante el trigger <code>trg_sync_role</code>. 4. El email debe ser único; se muestra error si ya existe en el sistema. 5. El nuevo usuario puede iniciar sesión inmediatamente con las credenciales asignadas. 6. El administrador que crea al usuario permanece en la pantalla de listado sin interrupciones tras confirmar. 7. No es posible crear un usuario sin asignar un rol explícito.			

Cuadro 2.15. *HU-13: Creación de nuevos usuarios*

HU-14 — Activar y desactivar usuario

HU-14	Activar y desactivar usuario		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero poder desactivar un usuario sin eliminarlo para gestionar bajas temporales del personal sin perder el historial de órdenes asociado.			
Criterios de aceptación 1. La lista de usuarios muestra un toggle o acción para activar/desactivar cada cuenta. 2. Al desactivar, el campo <code>active=false</code> se actualiza en la tabla <code>users</code>. 3. Un usuario inactivo no puede iniciar sesión en el sistema. 4. El historial de órdenes del usuario inactivo se conserva intacto en la base de datos. 5. El administrador recibe confirmación visual tras el cambio de estado. 6. No es posible desactivar la propia cuenta del administrador que está en sesión.			

Cuadro 2.16. HU-14: Activación y desactivación de usuarios

2.5.6 Gestión de mesas (admin)

HU-15 — CRUD de mesas del restaurante

HU-15	CRUD de mesas del restaurante		
Estado	Sin empezar	Asignado	—
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero crear, editar y desactivar mesas para reflejar la distribución real del restaurante en el sistema.			
Criterios de aceptación 1. Se puede crear una mesa indicando número y label descriptivo opcional. 2. El número de mesa es único dentro del restaurante; se muestra error si se intenta duplicar. 3. El label de una mesa existente puede editarse sin recrearla. 4. Las mesas pueden activarse y desactivarse; las inactivas no aparecen en las vistas operativas del mesero. 5. Las mesas con órdenes activas no pueden desactivarse hasta que la orden sea cerrada. 6. La lista de mesas muestra número, label y estado (activa/inactiva) de forma clara. 7. Las operaciones de creación y edición son accesibles únicamente para el rol admin.			

Cuadro 2.17. HU-15: CRUD de mesas del restaurante

HU-16 — Gestionar restaurante y su slug

HU-16	Gestionar restaurante y su slug		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero configurar el nombre y slug de mi restaurante para que las URLs públicas de la carta digital y sesiones de cliente sean reconocibles y únicas.			
Criterios de aceptación 1. La pantalla de configuración permite editar el nombre y slug del restaurante. 2. El slug es validado para ser único en la tabla restaurants; se muestra error si ya está en uso. 3. El slug admite solo letras minúsculas, números y guiones; se rechaza cualquier otro carácter. 4. El slug se usa automáticamente en las rutas <code>/menu/[slug]</code> y <code>/table/[slug]/[tableId]</code>. 5. Al cambiar el slug, se muestra una advertencia de que las URLs públicas anteriores dejarán de funcionar. 6. Solo el administrador puede acceder y modificar la configuración del restaurante.			

Cuadro 2.18. *HU-16: Configuración del restaurante y slug*

2.5.7 Gestión de menú (admin)

HU-17 — Listar ítems del menú con filtro por categoría

HU-17	Listar ítems del menú con filtro por categoría		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero ver todos los platos del menú organizados y con capacidad de filtrar por categoría para tener visibilidad y control del catálogo completo.			
Criterios de aceptación 1. La lista muestra nombre, precio, categoría y estado de disponibilidad de cada ítem. 2. El filtro por categoría reduce la lista a los ítems de la categoría seleccionada. 3. Los ítems no disponibles se diferencia visualmente de los disponibles. 4. La lista incluye tanto ítems disponibles como no disponibles en la vista del admin. 5. Desde cada ítem se puede acceder directamente a su formulario de edición. 6. Solo el administrador puede acceder a esta pantalla.			

Cuadro 2.19. HU-17: Listado de ítems del menú

HU-18 — Crear ítem del menú con imagen

HU-18	Crear ítem del menú con imagen		
Estado	Sin empezar	Asignado	—
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero agregar nuevos platos al menú con foto, descripción y precio para mantener el catálogo actualizado y atractivo para los clientes.			
Criterios de aceptación 1. El formulario incluye campos de nombre, descripción, precio, categoría y disponibilidad. 2. La imagen se selecciona desde la galería del dispositivo y se sube al bucket <code>menu-ings</code> de Supabase Storage. 3. La URL pública del CDN se guarda en el campo <code>image_url</code> del ítem creado. 4. El campo precio acepta únicamente valores numéricos positivos con hasta dos decimales. 5. El nuevo ítem aparece de inmediato en el listado del menú tras confirmarse la creación. 6. Si la subida de la imagen falla, el formulario muestra el error sin perder los datos ya ingresados. 7. El ítem se crea con <code>available=true</code> por defecto salvo que el admin lo deshabilite explícitamente.			

Cuadro 2.20. HU-18: Creación de ítem del menú

HU-19 — Editar precio y descripción de un ítem

HU-19		Editar precio y descripción de un ítem	
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero poder actualizar el precio o la descripción de un plato existente sin tener que eliminarlo y recrearlo.			
Criterios de aceptación 1. El formulario de edición llega pre-poblado con los datos actuales del ítem. 2. Los campos editables incluyen nombre, descripción, precio, categoría e imagen. 3. Al confirmar, el campo <code>updated_at</code> se actualiza automáticamente a <code>now()</code>. 4. El cambio de precio no afecta las órdenes anteriores, ya que el precio se guarda como snapshot en <code>order_items.unit_price</code>. 5. El cambio se refleja de inmediato en el listado del menú y en la carta digital pública. 6. Si el usuario cancela la edición, no se persiste ningún cambio en la base de datos.			

Cuadro 2.21. *HU-19: Edición de ítem del menú*

HU-20 — Activar o desactivar disponibilidad de un ítem

HU-20		Activar o desactivar disponibilidad de un ítem	
Estado	Sin empezar	Asignado	—
Esfuerzo	1 punto	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero marcar un plato como no disponible al instante cuando se agota para que los meseros y clientes no puedan pedirlo.			
Criterios de aceptación 1. El toggle de disponibilidad está visible en el listado de menú sin necesidad de entrar al formulario completo. 2. Al desactivar, el campo <code>available=false</code> se actualiza inmediatamente en la base de datos. 3. Un ítem con <code>available=false</code> no aparece en la carta pública ni en la vista del mesero al tomar pedidos. 4. Un ítem con <code>available=false</code> sí aparece en la vista del menú del cliente en mesa, marcado visualmente como no disponible. 5. La operación puede revertirse en cualquier momento activando el toggle nuevamente. 6. El cambio se refleja en la carta digital sin necesidad de recargar la página.			

Cuadro 2.22. *HU-20: Control de disponibilidad de ítem del menú*

HU-21 — Eliminar ítem del menú

HU-21	Eliminar ítem del menú		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero eliminar platos que ya no se ofrecen en el restaurante para mantener el menú limpio y libre de entradas obsoletas.			
Criterios de aceptación 1. Se muestra un diálogo de confirmación antes de ejecutar la eliminación. 2. Si el ítem tiene <code>order_items</code> asociados, el sistema lo desactiva (<code>available=false</code>) en lugar de eliminarlo, preservando la integridad del historial. 3. Si el ítem no tiene órdenes asociadas, la eliminación procede y el registro se borra de la base de datos. 4. La imagen asociada en Supabase Storage también se elimina cuando el registro se borra físicamente. 5. El ítem desaparece del listado del menú y de la carta digital inmediatamente tras la eliminación. 6. El administrador recibe retroalimentación sobre si el ítem fue eliminado o solo desactivado.			

Cuadro 2.23. *HU-21: Eliminación de ítem del menú*

2.5.8 Toma de pedidos (mesero)

HU-22 — Ver el menú disponible al tomar un pedido

HU-22	Ver el menú disponible al tomar un pedido		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero ver el menú con los platos disponibles agrupados por categoría para seleccionarlos rápidamente al registrar un pedido.			
Criterios de aceptación 1. La vista muestra únicamente ítems con <code>available=true</code>. 2. Los ítems están agrupados por categoría con un encabezado visible por cada grupo. 3. Cada ítem muestra nombre, precio e imagen en miniatura. 4. La vista no muestra el botón de gestión ni opciones de edición. 5. La carga del menú no requiere recargar la pantalla si fue actualizado recientemente. 6. Los datos se obtienen desde el hook <code>useMenu</code> que aplica la política RLS correspondiente.			

Cuadro 2.24. *HU-22: Vista del menú disponible para el mesero*

HU-23 — Crear una nueva orden seleccionando mesa e ítems

HU-23		Crear una nueva orden seleccionando mesa e ítems	
Estado	Sin empezar	Asignado	—
Esfuerzo	5 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero crear una orden eligiendo una mesa libre y agregando ítems con cantidades para registrar el pedido del cliente de forma rápida y precisa.			
Criterios de aceptación 1. El formulario permite seleccionar únicamente mesas sin órdenes activas. 2. Es posible agregar múltiples ítems con cantidad variable al mismo pedido. 3. Cada ítem permite agregar una nota opcional (ej.: sin picante, sin cebolla). 4. El campo <code>unit_price</code> de cada <code>order_item</code> se guarda como snapshot del precio actual, no como referencia dinámica. 5. El <code>total</code> de la orden se calcula automáticamente como suma de los subtotales. 6. La orden se crea con <code>status='pending'</code> y <code>payment_status='pending'</code> por defecto. 7. Si la mesa seleccionada ya tiene una orden activa (race condition), el sistema muestra un error y no crea la orden duplicada. 8. El mesero queda como <code>waiter_id</code> de la orden recién creada.			

Cuadro 2.25. *HU-23: Creación de nueva orden*

HU-24 — Ver detalle de una orden activa

HU-24	Ver detalle de una orden activa		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero ver los ítems, el total y el estado actual de una orden activa para saber exactamente qué pidió la mesa y en qué punto está.			
Criterios de aceptación 1. La pantalla muestra la lista de <code>order_items</code> con nombre, cantidad, precio unitario y subtotal. 2. El total de la orden es visible y corresponde a la suma de los subtotales. 3. El estado actual de la orden (<code>pending</code>, <code>ready</code>, <code>delivered</code>) es visible de forma destacada. 4. Las notas por ítem se muestran si existen. 5. El método de pago registrado es visible en el detalle. 6. El mesero puede acceder al detalle únicamente de sus propias órdenes activas.			

Cuadro 2.26. HU-24: Detalle de orden activa

HU-25 — Agregar ítems a una orden existente

HU-25		Agregar ítems a una orden existente	
Estado	Sin empezar	Asignado	—
Esfuerzo	3 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero poder agregar más platos a una orden pendiente si el cliente solicita algo extra durante el servicio.			
Criterios de aceptación 1. La opción de agregar ítems solo está disponible cuando la orden tiene status='pending'. 2. Los nuevos order_items se insertan con el snapshot del precio vigente al momento de agregar. 3. El total de la orden se recalcula y actualiza automáticamente tras agregar ítems. 4. No es posible agregar ítems a una orden con status='ready' o superior. 5. La operación es accesible desde el detalle de la orden con un botón claramente identificado. 6. Los nuevos ítems se reflejan inmediatamente en el detalle de la orden sin recargar la pantalla.			

Cuadro 2.27. *HU-25: Adición de ítems a una orden existente*

HU-26 — Marcar una orden como entregada

HU-26	Marcar una orden como entregada		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero marcar una orden como entregada una vez que llevé los platos a la mesa para liberar el estado de la mesa y cerrar el ciclo del pedido.			
Criterios de aceptación 1. La acción de marcar como entregada solo está disponible cuando la orden tiene <code>status='ready'</code>. 2. Al confirmar, el status transiciona de <code>ready</code> a <code>delivered</code>. 3. El trigger <code>trg_set_closed_at</code> asigna <code>closed_at</code> automáticamente en la misma transacción. 4. La mesa pasa a estado libre en la vista del mesero inmediatamente después. 5. La orden entregada ya no aparece en la lista de órdenes activas del mesero. 6. Se muestra un diálogo de confirmación antes de ejecutar la transición para evitar errores.			

Cuadro 2.28. *HU-26: Marcado de orden como entregada*

HU-27 — Cancelar una orden

HU-27	Cancelar una orden		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero o administrador, quiero poder cancelar una orden por error o solicitud del cliente para que la mesa quede libre y el pedido no se contabilice.			
Criterios de aceptación 1. La cancelación solo es posible cuando la orden tiene status='pending'. 2. Al confirmar, el status transiciona a cancelled. 3. La mesa queda libre en la vista del mesero inmediatamente después de la cancelación. 4. Las órdenes canceladas no aparecen en reportes de ingresos. 5. Se muestra un diálogo de confirmación antes de ejecutar la cancelación. 6. El mesero solo puede cancelar sus propias órdenes; el admin puede cancelar cualquiera. 7. La orden cancelada permanece en la base de datos con su historial de ítems intacto.			

Cuadro 2.29. HU-27: Cancelación de orden

HU-28 — Seleccionar método de pago al crear la orden

HU-28		Seleccionar método de pago al crear la orden	
Estado	Sin empezar	Asignado	—
Esfuerzo	1 punto	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero indicar si el cliente pagará en efectivo o por QR al registrar el pedido para facilitar el posterior proceso de cierre de la venta.			
Criterios de aceptación 1. El formulario de nueva orden incluye un selector con las opciones <code>cash</code> y <code>qr</code> . 2. El método de pago seleccionado se guarda en <code>orders.payment_method</code> . 3. No es posible confirmar la creación de la orden sin seleccionar un método de pago. 4. El método de pago seleccionado es visible en el detalle de la orden. 5. Para órdenes con <code>payment_method='cash'</code> , el <code>payment_status</code> puede confirmarse sin intervención adicional del admin. 6. Para órdenes con <code>payment_method='qr'</code> , el <code>payment_status</code> permanece <code>pending</code> hasta que el admin lo confirme.			

Cuadro 2.30. HU-28: Selección de método de pago

2.5.9 Pagos (admin)

HU-29 — Ver órdenes con pago QR pendiente de confirmación

HU-29	Ver órdenes con pago QR pendiente de confirmación		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero ver todas las órdenes con pago QR pendiente de verificación para poder confirmarlos manualmente y que queden registrados en los reportes.			
Criterios de aceptación 1. La lista filtra órdenes con <code>payment_method='qr'</code> y <code>payment_status='pending'</code>. 2. Cada entrada muestra número de mesa, total y hora de creación de la orden. 3. La lista se actualiza en tiempo real cuando una nueva orden QR entra al sistema. 4. Las órdenes ya confirmadas desaparecen de esta lista. 5. Solo el administrador puede acceder a esta pantalla. 6. La pantalla muestra un estado vacío descriptivo cuando no hay pagos QR pendientes.			

Cuadro 2.31. *HU-29: Listado de pagos QR pendientes*

HU-30 — Confirmar pago QR de una orden

HU-30	Confirmar pago QR de una orden		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero confirmar el pago QR de una orden para que el ingreso quede registrado en los reportes financieros y haya trazabilidad del cobro.			
Criterios de aceptación 1. Al confirmar, el <code>payment_status</code> transiciona de <code>pending</code> a <code>confirmed</code>. 2. Se inserta un registro en la tabla <code>payments</code> con el <code>amount</code>, <code>method='qr'</code> y <code>confirmed_by</code> con el id del admin. 3. La orden confirmada pasa a ser contabilizable en los reportes financieros. 4. Solo el administrador puede ejecutar esta acción. 5. Se muestra un diálogo de confirmación antes de ejecutar el cambio. 6. La orden desaparece de la lista de pagos QR pendientes tras la confirmación. 7. El campo <code>paid_at</code> en <code>payments</code> se registra con la marca de tiempo exacta de la confirmación.			

Cuadro 2.32. *HU-30: Confirmación de pago QR*

HU-31 — Mostrar QR estático de pago al cliente

HU-31		Mostrar QR estático de pago al cliente	
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero mostrar el QR del banco o billetera digital al cliente directamente desde la app para agilizar el cobro sin necesidad de un dispositivo adicional.			
Criterios de aceptación 1. El administrador puede subir la imagen del QR de pago desde la configuración del restaurante. 2. El mesero puede abrir el QR de pago en pantalla completa desde el detalle de la orden. 3. La imagen del QR se muestra con alta resolución para facilitar el escaneo del cliente. 4. El QR se muestra solo cuando la orden tiene <code>payment_method='qr'</code>. 5. La acción de mostrar el QR no modifica ningún estado de la orden. 6. Si no hay QR configurado, el mesero ve un mensaje indicando que debe configurarlo el admin.			

Cuadro 2.33. *HU-31: Visualización del QR de pago*

2.5.10 Gastos operativos (admin)

HU-32 — Registrar un gasto operativo

HU-32	Registrar un gasto operativo		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero registrar los gastos del día como insumos y pagos a proveedores para tener el egreso real reflejado en los reportes financieros.			
Criterios de aceptación 1. El formulario incluye campos de tipo de gasto, monto, descripción y fecha. 2. El monto solo acepta valores numéricos positivos con hasta dos decimales. 3. El gasto se guarda en la tabla <code>expenses</code> con el <code>registered_by</code> del admin en sesión. 4. Solo el administrador puede insertar gastos, validado por la política RLS <code>admin_insert_expenses</code>. 5. Tras guardar, el gasto aparece de inmediato en el listado de egresos. 6. El campo fecha permite registrar gastos de días anteriores, no solo el día actual.			

Cuadro 2.34. *HU-32: Registro de gasto operativo*

HU-33 — Listar gastos por rango de fechas

HU-33	Listar gastos por rango de fechas		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero ver los gastos registrados en un período determinado para revisar el historial de egresos y compararlo con los ingresos del mismo periodo.			
Criterios de aceptación 1. La pantalla incluye un selector de fecha desde y fecha hasta. 2. La lista resultante muestra tipo, monto, descripción y fecha de cada gasto. 3. El total acumulado del período es visible al final de la lista. 4. El filtro se aplica sobre el campo <code>expense_date</code> de la tabla <code>expenses</code>. 5. La lista se ordena cronológicamente por fecha de gasto de forma descendente. 6. La pantalla muestra un estado vacío si no hay gastos en el período seleccionado.			

Cuadro 2.35. *HU-33: Listado de gastos por período***HU-34 — Editar y eliminar un gasto**

HU-34	Editar y eliminar un gasto		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero corregir o eliminar un gasto mal registrado para mantener los egresos precisos y que los reportes reflejen la realidad financiera del restaurante.			
Criterios de aceptación 1. El formulario de edición llega pre-poblado con los datos actuales del gasto. 2. Todos los campos del gasto son editables: tipo, monto, descripción y fecha. 3. Se muestra un diálogo de confirmación antes de ejecutar la eliminación. 4. Tras eliminar, el gasto desaparece del listado y el total del período se recalcula. 5. Solo el administrador puede editar o eliminar gastos. 6. Al cancelar la edición, no se persiste ningún cambio en la base de datos.			

Cuadro 2.36. *HU-34: Edición y eliminación de gastos*

2.5.11 Vista de mesas en tiempo real (mesero)

HU-35 — Ver estado de todas las mesas asignadas

HU-35	Ver estado de todas las mesas asignadas		
Estado	Sin empezar	Asignado	—
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero ver el estado actual de cada una de mis mesas en una sola pantalla para saber dónde necesito actuar sin tener que revisar cada orden individualmente.			
Criterios de aceptación 1. La pantalla muestra tarjetas por mesa con los estados: Libre, Pendiente, Listo para entregar y Entregado. 2. El estado de cada mesa se deriva del campo status de la orden activa asociada. 3. Las mesas sin orden activa se muestran con estado Libre. 4. Esta pantalla es la pantalla principal del rol waiter al iniciar sesión. 5. El componente TableCard representa cada mesa de forma visual diferenciada por estado. 6. El mesero solo ve las mesas que tienen órdenes asignadas a él o mesas del restaurante sin orden activa. 7. La pantalla muestra el número y label de cada mesa.			

Cuadro 2.37. HU-35: Vista de estado de mesas del mesero

HU-36 — Actualizar estado de mesas en tiempo real sin recargar

HU-36		Actualizar estado de mesas en tiempo real sin recargar	
Estado	Sin empezar	Asignado	—
Esfuerzo	4 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero que el estado de mis mesas se actualice automáticamente cuando cambie para no tener que refrescar la pantalla durante el servicio.			
Criterios de aceptación 1. El hook <code>useTables</code> se suscribe al canal <code>orders</code> de Supabase Realtime filtrado por <code>waiter_id</code>. 2. Al recibir un evento de cambio en <code>orders</code>, el estado visual de la mesa afectada se actualiza sin recargar la pantalla. 3. La suscripción se activa al montar el componente y se cancela correctamente al desmontarlo. 4. El estado de la suscripción (conectado/desconectado) se maneja sin exponer errores al usuario en condiciones normales. 5. La actualización en UI ocurre en menos de un segundo tras el evento de base de datos. 6. El hook es reutilizable y no genera suscripciones duplicadas si el componente se re-renderiza.			

Cuadro 2.38. *HU-36: Actualización en tiempo real del estado de mesas*

HU-37 — Recibir notificación cuando un pedido está listo

HU-37		Recibir notificación cuando un pedido está listo	
Estado	Sin empezar	Asignado	—
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero recibir una notificación visible en la app cuando mi pedido pasa a estado listo para ir a buscarlo a cocina sin tener que estar revisando constantemente la pantalla.			
Criterios de aceptación 1. Al recibir un evento UPDATE en <code>orders</code> donde <code>status='ready'</code>, se dispara una alerta visible en pantalla. 2. La alerta muestra el número de mesa del pedido listo. 3. La tarjeta de la mesa correspondiente se resalta visualmente con el estado Listo para entregar. 4. La notificación se muestra incluso si el mesero está en otra pantalla de la app. 5. La alerta se descarta manualmente o desaparece tras un tiempo configurable. 6. Solo se notifica al mesero propietario de la orden, no a todos los meseros.			

Cuadro 2.39. *HU-37: Notificación de pedido listo*

HU-38 — Manejar reconexión automática del canal Realtime

HU-38	Manejar reconexión automática del canal Realtime		
Estado	Sin empezar	Asignado	—
Esfuerzo	5 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero que la app se reconecte automáticamente al canal en tiempo real si pierde la conexión para no perder notificaciones críticas durante el servicio.			
Criterios de aceptación 1. Los eventos <code>CHANNEL_ERROR</code> y <code>TIMED_OUT</code> de Supabase Realtime disparan la lógica de reconexión. 2. La reconexión implementa un mecanismo de backoff exponencial para no saturar el servidor. 3. La UI muestra un indicador visual cuando la conexión Realtime está interrumpida. 4. Cuando se restablece la conexión, el indicador desaparece y la suscripción se reanuda. 5. Tras una reconexión, el estado de las mesas se refresca desde la base de datos para corregir posibles eventos perdidos. 6. La lógica de reconexión está encapsulada en una función <code>scheduleReconnect</code> reutilizable. 7. Los intentos de reconexión se detienen si el componente es desmontado antes de completarse.			

Cuadro 2.40. *HU-38: Reconexión automática al canal Realtime*

2.5.12 Llamada al mesero (cliente → mesero)

HU-39 — Mostrar botón “Llamar al mesero” en la sesión de cliente

HU-39			
Mostrar botón “Llamar al mesero” en la sesión de cliente			
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como cliente en mesa, quiero poder llamar al mesero desde mi celular con un botón para no tener que levantarme ni interrumpir la dinámica del restaurante.			
Criterios de aceptación 1. El botón <code>CallWaiterButton</code> es visible en la ruta <code>/table/[slug]/[tableId]</code>. 2. Al presionarlo, se inserta un registro en <code>table_calls</code> con el <code>table_id</code> de la sesión anónima activa. 3. La política RLS <code>customer_insert_table_call</code> valida que el <code>table_id</code> del JWT coincida con el de la inserción. 4. El botón muestra un estado de carga mientras se procesa la inserción. 5. Tras llamar exitosamente, el botón muestra una confirmación y se deshabilita temporalmente para evitar llamadas duplicadas. 6. Si no hay sesión anónima activa, el botón no es visible ni funcional. 7. El cliente no puede llamar al mesero si la mesa no tiene una sesión iniciada válida.			

Cuadro 2.41. HU-39: Botón de llamada al mesero desde el cliente

HU-40 — Recibir notificación de llamada de cliente en tiempo real

HU-40	Recibir notificación de llamada de cliente en tiempo real		
Estado	Sin empezar	Asignado	—
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero recibir una notificación inmediata cuando un cliente en mi mesa me llama para poder atenderlo rápidamente sin que tenga que esperar.			
Criterios de aceptación 1. El hook <code>useTableCalls</code> se suscribe al canal <code>table_calls</code> escuchando eventos <code>INSERT</code>. 2. Al recibir un evento, se muestra una alerta con el número de la mesa que llama. 3. La mesa correspondiente se resalta visualmente con el estado <code>Cliente llama</code> en la vista de mesas. 4. El mesero recibe la notificación incluso si está en otra pantalla de la app. 5. Solo el mesero con una orden activa en esa mesa recibe la notificación; otros meseros no son alertados. 6. La alerta es descartable manualmente por el mesero.			

Cuadro 2.42. *HU-40: Notificación de llamada de cliente en tiempo real*

HU-41 — Marcar llamada de cliente como atendida

HU-41		Marcar llamada de cliente como atendida	
Estado	Sin empezar	Asignado	—
Esfuerzo	1 punto	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como mesero, quiero poder marcar una llamada de cliente como atendida para que la alerta desaparezca y la mesa vuelva a su estado operativo normal.			
Criterios de aceptación 1. Al marcar como atendida, se ejecuta <code>UPDATE table_calls SET attended=true, attended_at=now()</code> . 2. El estado visual Cliente llama de la mesa se limpia en la vista de mesas. 3. La acción está disponible desde la alerta de notificación recibida. 4. La operación solo la puede ejecutar el mesero con la política RLS <code>waiter_table_calls</code> activa. 5. El campo <code>attended_at</code> queda registrado con la marca de tiempo exacta de la atención. 6. Tras marcar la llamada, la mesa regresa al estado que corresponda según su orden activa.			

Cuadro 2.43. *HU-41: Marcado de llamada de cliente como atendida*

2.5.13 Carta digital pública (web)

HU-42 — Publicar carta digital accesible por URL sin login

HU-42	Publicar carta digital accesible por URL sin login		
Estado	Sin empezar	Asignado	—
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como cliente del restaurante, quiero poder ver el menú escaneando un QR desde mi celular sin instalar ninguna app ni crear una cuenta.			
Criterios de aceptación 1. La ruta <code>/menu/[slug]</code> renderiza en el navegador usando el cliente <code>supabasePublic</code> con la anon key. 2. Solo se muestran ítems con <code>available=true</code>, aplicando la política <code>public_menu_read</code>. 3. La página es accesible sin autenticación desde cualquier navegador moderno. 4. No se requiere descarga de ninguna aplicación para visualizar la carta. 5. El slug del restaurante define qué menú se muestra, permitiendo múltiples restaurantes en el mismo sistema. 6. La página carga en menos de 3 segundos en condiciones normales de red. 7. Un slug inexistente muestra una página de error descriptiva, no un error técnico.			

Cuadro 2.44. HU-42: Carta digital pública

HU-43 — Mostrar carta organizada por categorías con imágenes y precios

HU-43	Mostrar carta organizada por categorías con imágenes y precios		
Estado	Sin empezar	Asignado	—
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como cliente, quiero ver el menú organizado con fotos y precios para decidir qué pedir de forma rápida y atractiva desde mi celular.			
Criterios de aceptación 1. Los ítems están agrupados por categoría con encabezados visibles. 2. Cada ítem muestra la imagen servida desde el CDN de Supabase Storage. 3. El precio de cada ítem es visible con formato monetario claro. 4. El diseño es responsive y está optimizado para dispositivos móviles. 5. Las imágenes se cargan de forma lazy para no bloquear la visualización inicial del menú. 6. Si un ítem no tiene imagen, se muestra un placeholder coherente con el diseño. 7. La carta es visualmente legible sin zoom adicional en pantallas de 5 pulgadas o más.			

Cuadro 2.45. HU-43: Presentación visual de la carta digital

HU-44 — Generar y mostrar QR del restaurante para carta pública

HU-44	Generar y mostrar QR del restaurante para carta pública		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero obtener el QR que apunta a la carta digital de mi restaurante para imprimirlo y colocarlo en la entrada o en las mesas para clientes que quieran consultar el menú.			
Criterios de aceptación 1. El QR se genera apuntando a la URL <code>/menu/[slug]</code> del restaurante. 2. El QR es descargable como imagen desde la pantalla de configuración del restaurante. 3. El QR es escaneable desde cualquier aplicación de cámara estándar en Android e iOS. 4. Si el slug del restaurante cambia, el QR se regenera automáticamente con la nueva URL. 5. El QR se muestra en resolución suficiente para ser impreso sin pixelarse. 6. Solo el administrador puede acceder a la sección de generación de QR.			

Cuadro 2.46. HU-44: Generación del QR de la carta pública

2.5.14 Sesión de cliente en mesa (web)

HU-45 — Iniciar sesión anónima al escanear el QR de la mesa

HU-45		Iniciar sesión anónima al escanear el QR de la mesa	
Estado	Sin empezar	Asignado	—
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como cliente en mesa, quiero que al escanear el QR de mi mesa se inicie automáticamente una sesión vinculada a esa mesa sin que tenga que registrarme ni hacer nada adicional.			
Criterios de aceptación 1. Al acceder a <code>/table/[slug]/[tableId]</code>, se invoca <code>signInAnonymously</code> con <code>role='customer'</code>, <code>table_id</code> y <code>restaurant_slug</code> en <code>user_metadata</code>. 2. La sesión anónima persiste durante la visita del cliente sin expirar prematuramente. 3. El cliente no necesita ingresar ningún dato ni crear una cuenta para iniciar la sesión. 4. Si ya existe una sesión anónima activa en el dispositivo, se reutiliza sin crear una nueva. 5. Un <code>tableId</code> inexistente muestra un mensaje de error descriptivo al cliente. 6. La sesión anónima no tiene acceso a ninguna funcionalidad de mesero ni administrador. 7. El proceso de inicio de sesión es transparente para el usuario; no muestra pantallas de carga prolongadas.			

Cuadro 2.47. HU-45: Sesión anónima del cliente en mesa

HU-46 — Ver menú completo desde la sesión de mesa

HU-46		Ver menú completo desde la sesión de mesa	
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como cliente en mesa, quiero ver todo el menú incluyendo platos no disponibles para saber qué hay y qué no hay hoy antes de decidir qué pedir.			
Criterios de aceptación 1. La política <code>customer_full_menu_read</code> permite la lectura de todos los ítems del menú, incluyendo no disponibles. 2. Los ítems no disponibles se muestran con un indicador visual claro (ej.: tachado o etiqueta Agotado). 3. Los ítems disponibles y no disponibles están diferenciados visualmente de forma inmediata. 4. El menú se organiza por categorías igual que en la carta pública. 5. El cliente no puede modificar ni interactuar con los ítems del menú, solo consultarlos. 6. La vista requiere que la sesión anónima esté activa; sin ella no carga el menú completo.			

Cuadro 2.48. *HU-46: Vista del menú completo desde la sesión de mesa*

HU-47 — Generar QR individual por mesa

HU-47	Generar QR individual por mesa		
Estado	Sin empezar	Asignado	—
Esfuerzo	2 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero obtener el QR individual de cada mesa para imprimirlo, laminarlo y colocarlo físicamente en la mesa correspondiente.			
Criterios de aceptación 1. Cada mesa en el listado de mesas muestra su propio QR apuntando a <code>/table/[slug]/[tableId]</code>. 2. El QR es descargable como imagen desde la fila de cada mesa en el listado. 3. El QR identifica unívocamente la mesa mediante su <code>tableId</code> en la URL. 4. El QR es escaneable desde cualquier cámara estándar en dispositivos Android e iOS. 5. Si el <code>tableId</code> o el slug cambian, el QR se regenera con la nueva URL. 6. Solo el administrador puede acceder y descargar los QR de mesas. 7. El QR se muestra en resolución suficiente para ser impreso sin pérdida de calidad.			

Cuadro 2.49. HU-47: Generación de QR por mesa

2.5.15 Reportes financieros (admin)

HU-48 — Crear función SQL `get_report` para ingresos y egresos

HU-48	Crear función SQL <code>get_report</code> para ingresos y egresos		
Estado	Sin empezar	Asignado	—
Esfuerzo	4 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero que los reportes financieros se calculen en el servidor mediante una función PostgreSQL para garantizar precisión, seguridad y rendimiento en cualquier volumen de datos.			
Criterios de aceptación 1. La función <code>get_report(date_from date, date_to date)</code> está creada en la migración <code>003_functions.sql</code>. 2. La función valida que el rol del invocante sea <code>admin</code> antes de ejecutar; lanza excepción si no lo es. 3. La función usa <code>SECURITY DEFINER</code> para acceder a tablas protegidas por RLS. 4. Retorna un objeto JSON con <code>gross_income</code>, <code>total_expenses</code> y <code>net_income</code>. 5. Solo contabiliza órdenes con <code>status='delivered'</code> y <code>payment_status='confirmed'</code>. 6. Los gastos se filtran por <code>expense_date</code> dentro del rango indicado. 7. La función es invocable desde la app mediante <code>supabase.rpc('get_report', {...})</code>. 8. Los valores retornados son correctos cuando no hay datos en el período (retorna ceros, no null).			

Cuadro 2.50. *HU-48: Función SQL para reporte financiero*

HU-49 — Crear función SQL `get_top_items` para ranking de platos

HU-49		Crear función SQL <code>get_top_items</code> para ranking de platos	
Estado	Sin empezar	Asignado	—
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como sistema, quiero tener una función SQL que devuelva los platos más vendidos por período para poblar el ranking en la aplicación del administrador.			
Criterios de aceptación 1. La función <code>get_top_items(date_from, date_to, item_limit)</code> está creada en la migración <code>003_functions.sql</code>. 2. Retorna una tabla con columnas <code>menu_item_id</code>, <code>name</code>, <code>total_qty</code> y <code>total_revenue</code>. 3. Los resultados se ordenan por <code>total_qty</code> de forma descendente. 4. Solo incluye ítems de órdenes con <code>status='delivered'</code> y <code>payment_status='confirmed'</code>. 5. El parámetro <code>item_limit</code> tiene valor por defecto de 10. 6. La función usa <code>SECURITY DEFINER</code> y debe ser envuelta en un validador de rol externo antes de llamarla. 7. Es invocable desde la app mediante <code>supabase.rpc('get_top_items', {...})</code>.			

Cuadro 2.51. *HU-49: Función SQL para ranking de platos*

HU-50 — Pantalla de reporte financiero por período

HU-50		Pantalla de reporte financiero por período	
Estado	Sin empezar	Asignado	—
Esfuerzo	3 puntos	Prioridad	Alta
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero ver los ingresos brutos, egresos y neto de un período seleccionado para entender la situación financiera del restaurante de forma rápida y clara.			
Criterios de aceptación 1. La pantalla incluye un selector de período con opciones: diario, semanal, mensual y rango libre. 2. Los datos se obtienen invocando <code>supabase.rpc('get_report', {date_from, date_to})</code>. 3. La pantalla muestra <code>gross_income</code>, <code>total_expenses</code> y <code>net_income</code> con formato monetario claro. 4. Cuando el <code>net_income</code> es negativo, se resalta visualmente como alerta. 5. La pantalla muestra un estado de carga mientras se obtienen los datos del servidor. 6. Solo el administrador puede acceder a esta pantalla; cualquier otro rol es redirigido. 7. Si no hay datos en el período seleccionado, se muestran ceros con un mensaje explicativo.			

Cuadro 2.52. *HU-50: Pantalla de reporte financiero*

HU-51 — Pantalla de ranking de platos más vendidos

HU-51	Pantalla de ranking de platos más vendidos		
Estado	Sin empezar	Asignado	—
Esfuerzo	3 puntos	Prioridad	Media
Versión	1.0	Última edición	2026-04-09
Descripción Como administrador, quiero ver qué platos se vendieron más en un período determinado para tomar decisiones informadas sobre el menú y las operaciones del restaurante.			
Criterios de aceptación 1. Los datos se obtienen invocando <code>supabase.rpc('get_top_items', {date_from, date_to, item_limit})</code>. 2. La lista muestra los platos ordenados por cantidad total vendida de forma descendente. 3. Cada entrada muestra nombre del plato, total de unidades vendidas y revenue generado. 4. La pantalla incluye un selector de período idéntico al de la pantalla de reporte financiero. 5. El límite de ítems mostrados es configurable, con valor por defecto de 10. 6. La pantalla muestra un estado de carga mientras se obtienen los datos del servidor. 7. Si no hay ventas en el período, se muestra un estado vacío descriptivo.			

Cuadro 2.53. *HU-51: Pantalla de ranking de platos más vendidos*

3. Diseño del Proyecto

3.1 Stack Tecnológico

Capa	Tecnología	Justificación
Mobile	React Native + Expo SDK 52+	Android, OTA updates vía EAS
Web	Expo Router (web)	Misma codebase, rutas públicas en browser
Lenguaje	TypeScript	Tipado estático, seguridad en desarrollo
BaaS	Supabase (free tier)	PostgreSQL + Auth + Storage + Realtime sin servidor
Cliente BaaS	supabase-js v2	SDK oficial, soporte completo
Base Datos	PostgreSQL (vía Supabase)	Modelo relacional, consistencia transaccional, RLS
Auth	Supabase Auth	JWT, roles vía <code>user_metadata</code> , sesiones anónimas
Storage	Supabase Storage	Bucket <code>menu-imgs</code> (CDN público)
Tiempo real	Supabase Realtime	Canal <code>orders</code> y <code>table_calls</code>
Reportes	PostgreSQL functions + RPC	Agregaciones server-side
Pagos QR	QR estático banco/billetera	Sin pasarela; verificación manual por admin
Deploy	Expo EAS Build + EAS Update	Build nativo y OTA sin pasar por stores
Versiones	Git + GitHub	Branching por feature, PRs obligatorios

Cuadro 3.1. *Stack tecnológico del sistema*

3.2 Modelo de Datos

3.2.1 Entidades y Relaciones

El sistema define las siguientes entidades principales y sus relaciones:

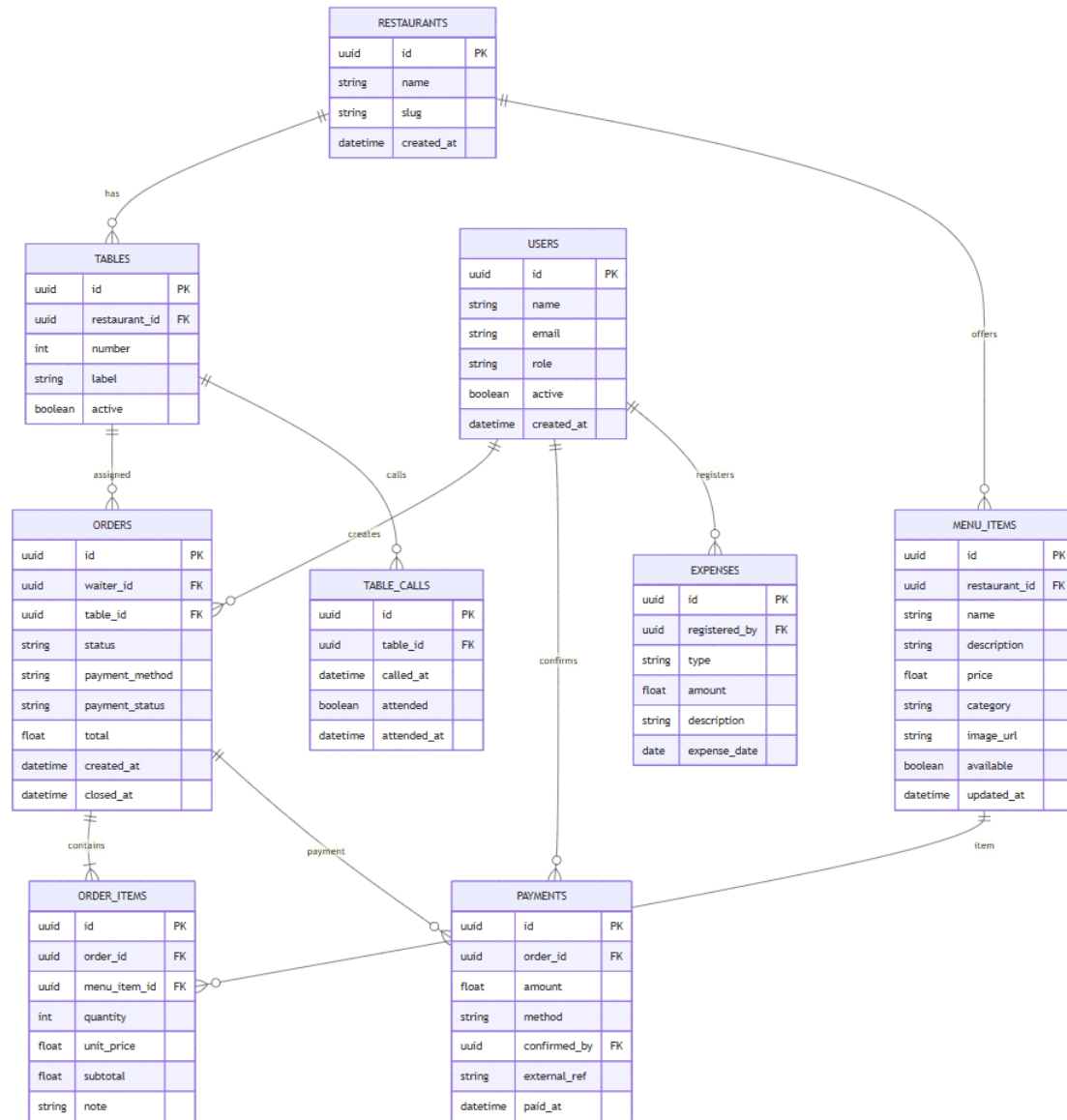


Figura 3.1. Modelo entidad-relación de la base de Datos

3.2.2 Esquema de Tablas

A continuación se describe cada tabla del modelo de datos:

restaurants

Campo	Tipo	Descripción
id	uuid	PK
name	text	Nombre del restaurante
slug	text UNIQUE	Identificador en URLs públicas
created_at	timestampz	Fecha de creación

tables

Campo	Tipo	Descripción
id	uuid	PK
restaurant_id	uuid FK	Referencia a restaurants
number	integer	Número de mesa
label	text	Nombre descriptivo (ej: "Terraza 3")
active	boolean	Estado activo/inactivo

users

Campo	Tipo	Descripción
id	uuid	PK (sincronizado con auth.users)
name	text	Nombre del usuario
email	text UNIQUE	Correo electrónico
role	enum	waiter admin
active	boolean	Estado de la cuenta
created_at	timestampz	Fecha de registro

Nota

password_hash se omite: Supabase Auth gestiona credenciales internamente. El campo **role** se sincroniza hacia **auth.users.raw_user_meta_data** mediante un trigger **AFTER UPDATE**.

menu_items

Campo	Tipo	Descripción
id	uuid	PK
restaurant_id	uuid FK	Referencia a restaurants
name	text	Nombre del platillo
description	text	Descripción
price	decimal(10,2)	Precio
category	text	Categoría
image_url	text	URL del CDN de Storage
available	boolean	Disponibilidad en tiempo real
updated_at	timestampz	Última actualización

orders

Campo	Tipo	Descripción
id	uuid	PK
waiter_id	uuid FK	Referencia a users
table_id	uuid FK	Referencia a tables
status	enum	pending ready delivered cancelled
payment_method	enum	cash qr
payment_status	enum	pending confirmed
total	decimal(10,2)	Total de la orden
created_at	timestamp tz	Fecha de apertura
closed_at	timestamp tz	Asignado automáticamente al entregar

order_items

Campo	Tipo	Descripción
id	uuid	PK
order_id	uuid FK	Referencia a orders
menu_item_id	uuid FK	Referencia a menu_items
quantity	integer	Cantidad
unit_price	decimal(10,2)	Snapshot del precio al momento del pedido
subtotal	decimal(10,2)	Subtotal
note	text	Indicación especial del cliente

payments

Campo	Tipo	Descripción
id	uuid	PK
order_id	uuid FK	Referencia a orders
amount	decimal(10,2)	Monto pagado
method	enum	cash qr
confirmed_by	uuid FK	Admin que confirmó el pago
external_ref	text	Reservado para pasarela futura
paid_at	timestamp tz	Fecha/hora del pago

expenses

Campo	Tipo	Descripción
id	uuid	PK
registered_by	uuid FK	Usuario que registró el gasto
type	text	Tipo de gasto
amount	decimal(10,2)	Monto
description	text	Descripción
expense_date	date	Fecha del gasto

table_calls

Campo	Tipo	Descripción
id	uuid	PK
table_id	uuid FK	Referencia a tables
called_at	timestampz	Momento de la llamada
attended	boolean	Si fue atendida
attended_at	timestampz	Momento de atención

3.3 Estructura del Proyecto

La estructura de directorios del repositorio sigue las convenciones de Expo Router:

```
gestion-comandas/
|-- app/
|   |-- (auth)/          login.tsx, _layout.tsx
|   |-- (waiter)/        tables/, orders/, _layout.tsx
|   |-- (admin)/         menu/, expenses/, reports/, _layout.tsx
|   |-- menu/[slug].tsx
|   |-- table/[slug]/[tableId].tsx
|   \-- _layout.tsx
|-- components/
|   |-- tables/TableCard.tsx
|   |-- orders/
|   |-- menu/
|   |-- customer/CallWaiterButton.tsx
|   \-- ui/
|-- lib/
|   |-- supabase.ts
|   |-- supabase-public.ts
|   |-- hooks/
|   |   |-- useOrders.ts
|   |   |-- useTables.ts
|   |   |-- useTableCalls.ts
|   |   |-- useMenu.ts
|   |   \-- useReports.ts
|   \-- types/database.types.ts
|-- supabase/
|   |-- migrations/
|   |   |-- 001_schema.sql
|   |   |-- 002_rls.sql
|   |   |-- 003_functions.sql
|   |   \-- 004_triggers.sql
|   \-- config.toml
|-- app.json
|-- eas.json
\-- tsconfig.json
```

3.4 Roles y Accesos

Funcionalidad	Waiter	Admin	Customer
Ver menú (solo disponibles)	✓	✓	carta pública
Ver menú completo (con no disp.)	✓	✓	✓ (anónimo)
Crear pedido	✓	✓	—
Actualizar estado de pedido	✓	✓	—
Confirmar pago QR	—	✓	—
Vista de estado de mesas	✓	—	—
Recibir notificación de llamada	✓	—	—
Llamar al mesero	—	—	✓
Gestionar menú (CRUD + imágenes)	—	✓	—
Registrar gastos	—	✓	—
Ver reportes financieros	—	✓	—
Ver ranking de platos más vendidos	—	✓	—
Gestionar usuarios	—	✓	—
Carta digital (pública, sin auth)	—	—	—

Cuadro 3.2. Matriz de roles y accesos del sistema

4. Arquitectura del Sistema

La arquitectura del sistema Gestión de Comandas adopta el modelo **BaaS (Backend as a Service)**. En este enfoque, el cliente móvil consume directamente los servicios proporcionados por Supabase sin la necesidad de un backend intermedio personalizado.

Supabase provee servicios administrados de base de datos, autenticación, almacenamiento de archivos y sincronización en tiempo real, lo cual reduce la complejidad de infraestructura y permite enfocar el desarrollo en la lógica de negocio y la experiencia de usuario.

4.1 Diagrama de Contexto

El diagrama de contexto muestra la interacción general entre los actores del sistema (mesero, administrador y cliente) y los servicios principales utilizados en la solución. Se observa cómo la aplicación móvil y la carta digital web se comunican directamente con Supabase mediante protocolos REST, WebSocket y mecanismos de autenticación basados en JWT.

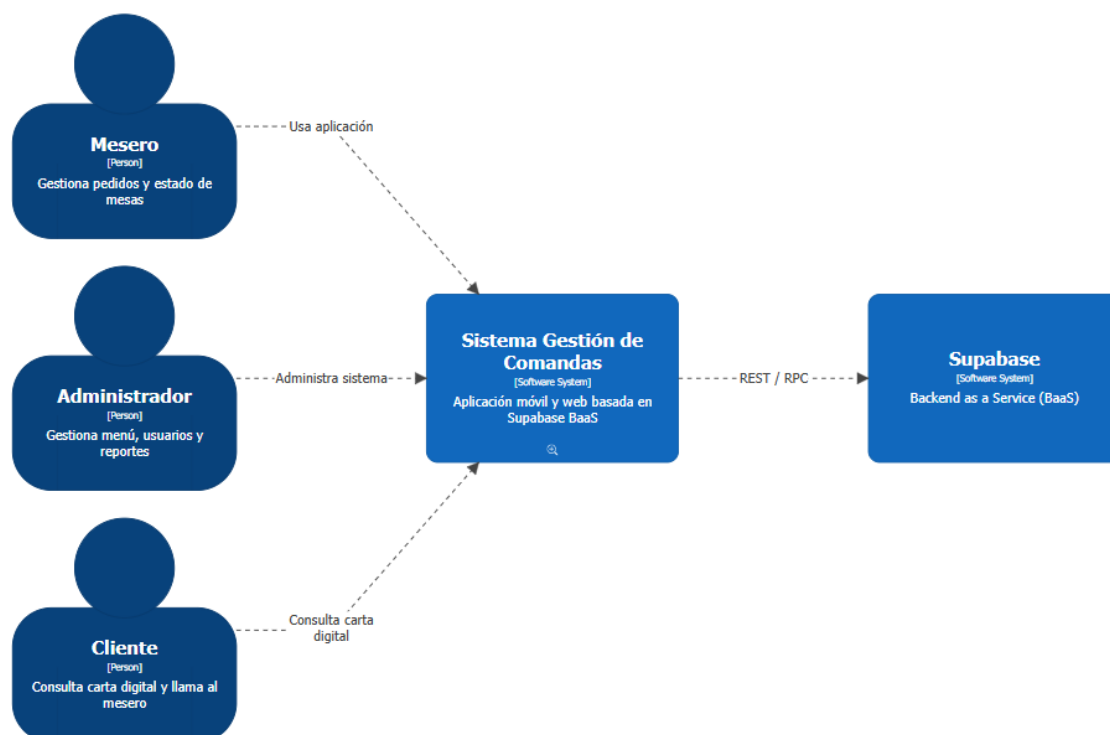


Figura 4.1. Diagrama de Contexto del sistema Gestión de Comandas basado en modelo BaaS

En este nivel se identifican los usuarios del sistema y los sistemas externos con los que interactúa la solución. El cliente móvil implementado en React Native consume directamente los servicios de Supabase PostgreSQL, Supabase Auth, Supabase Storage y Supabase Realtime. Asimismo, la carta digital accesible mediante navegador web permite a los clientes consultar el menú y solicitar atención sin necesidad de autenticación tradicional.

4.2 Diagrama de Contenedores

El diagrama de contenedores describe los bloques principales que componen la solución y cómo estos se comunican entre sí. Se distinguen claramente los contenedores del cliente móvil, las rutas web públicas y los servicios gestionados por Supabase.

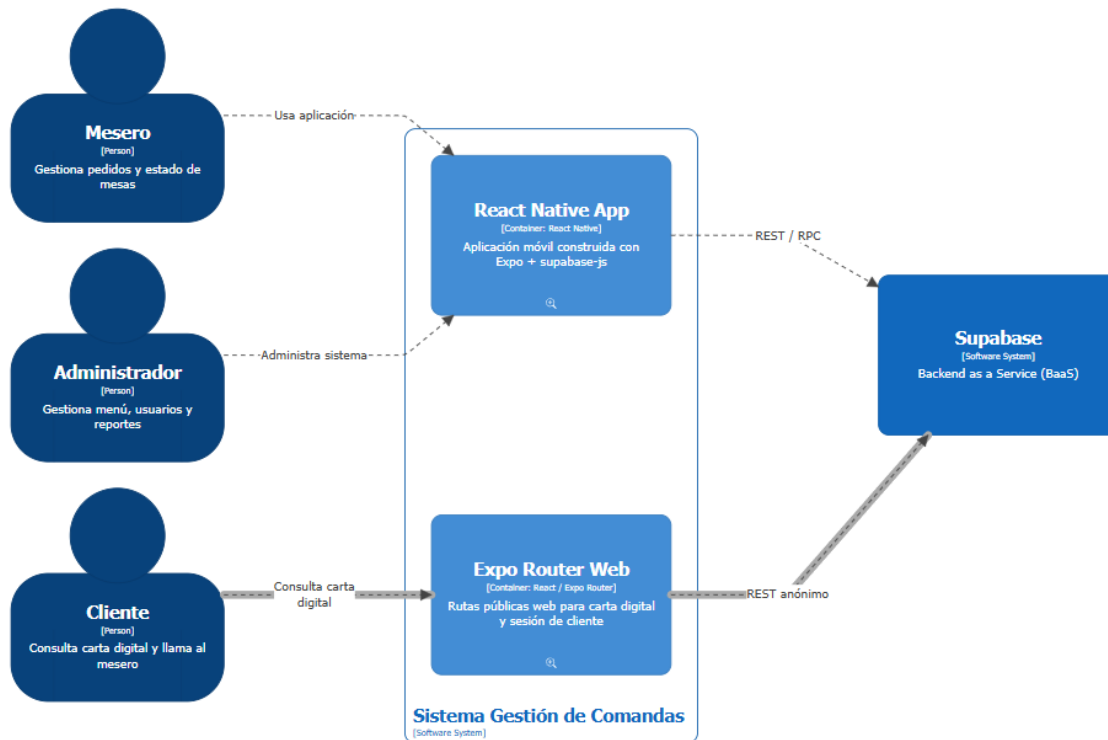


Figura 4.2. *Diagrama de Contenedores del sistema utilizando Supabase como Backend as a Service*

La aplicación móvil desarrollada con React Native y Expo se conecta directamente a Supabase mediante el SDK oficial supabase-js. Este cliente permite realizar consultas a la base de datos, autenticación de usuarios, subida de archivos y suscripción a eventos en tiempo real.

Por otro lado, Expo Router permite exponer rutas web públicas que facilitan la visualización de la carta digital y la sesión del cliente en mesa mediante el uso de códigos QR. Estas rutas utilizan acceso anónimo controlado mediante las políticas de seguridad Row Level Security (RLS).

Supabase actúa como proveedor de Backend as a Service ofreciendo:

- Base de datos PostgreSQL con funciones SQL y políticas RLS.
- Servicio de autenticación basado en JWT y roles.
- Almacenamiento de imágenes del menú mediante Storage con acceso público.
- Canales de comunicación en tiempo real mediante WebSocket.

4.3 Diagrama de Componentes — Funcionalidades Clave

4.3.1 Componentes de la aplicación móvil

El siguiente diagrama muestra los componentes principales de la aplicación móvil utilizada por el mesero y el administrador. Estos componentes permiten gestionar pedidos, visualizar el estado de mesas, administrar el menú y consultar reportes financieros.

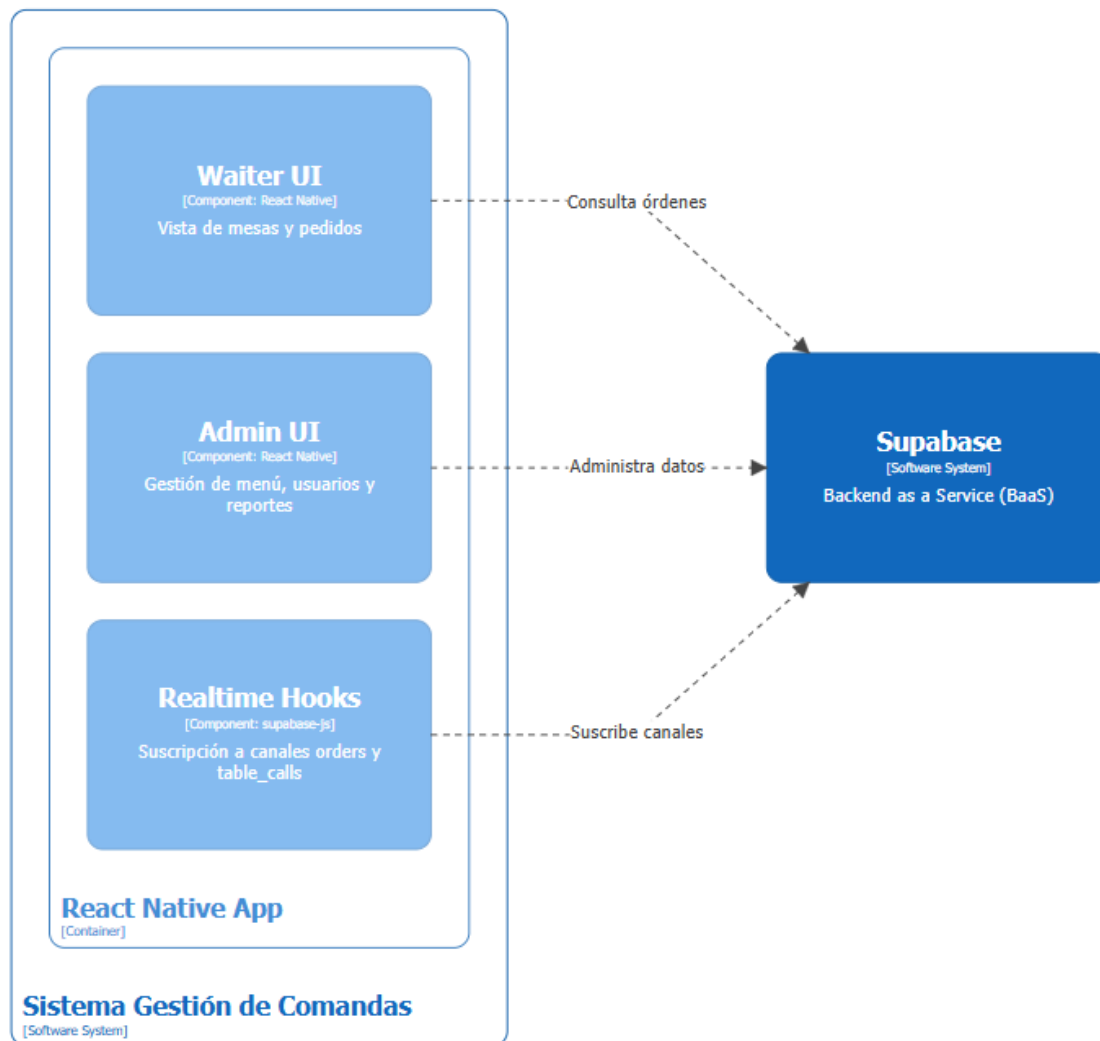


Figura 4.3. Componentes principales de la aplicación React Native

Los componentes Waiter UI y Admin UI representan las interfaces de usuario según el rol autenticado. El módulo Realtime Hooks permite la suscripción a los canales de Supabase Realtime para actualizar el estado de las mesas y los pedidos sin necesidad de recargar la aplicación.

4.3.2 Componentes de rutas web públicas

El siguiente diagrama muestra los componentes asociados a la carta digital y la sesión del cliente en mesa. Estas funcionalidades permiten que el cliente visualice el menú mediante un código QR y pueda solicitar atención al mesero.

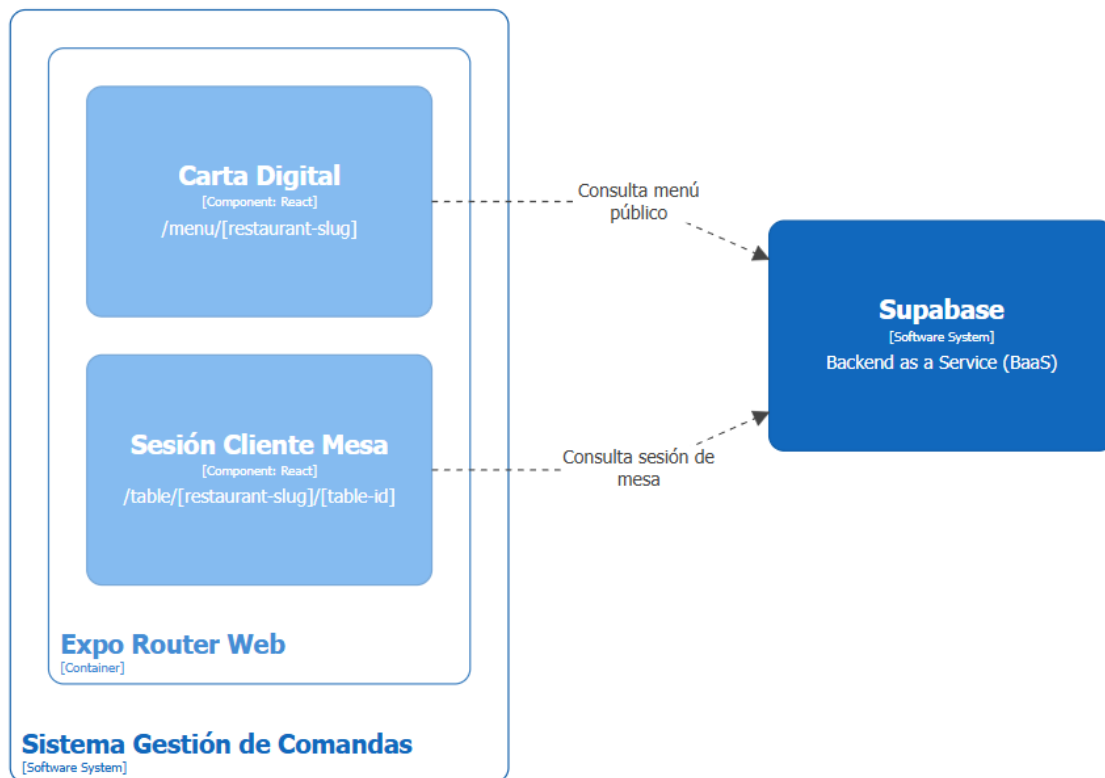


Figura 4.4. Componentes de la carta digital y sesión de cliente en mesa

Las rutas públicas permiten acceder al menú del restaurante utilizando un identificador único (slug). La sesión de mesa permite identificar la mesa desde la cual el cliente solicita atención, registrando eventos en la base de datos que son posteriormente consumidos por la aplicación del mesero en tiempo real.

4.3.3 Contexto de Supabase

El siguiente diagrama muestra cómo la aplicación cliente interactúa con los servicios ofrecidos por Supabase dentro del modelo Backend as a Service (BaaS). En este enfoque, el cliente móvil y las rutas web públicas consumen directamente los servicios de autenticación, base de datos, almacenamiento y comunicación en tiempo real sin necesidad de un backend intermedio personalizado.



Figura 4.5. Relación entre la aplicación cliente y los servicios de Supabase

Supabase proporciona un conjunto de servicios integrados que permiten reducir la complejidad de infraestructura. El SDK supabase-js facilita la conexión desde la aplicación

React Native hacia PostgreSQL, Auth, Storage y Realtime utilizando protocolos seguros basados en JWT y WebSocket.

4.3.4 Contenedores de Supabase

El diagrama de contenedores presenta los servicios principales que conforman el Backend as a Service utilizado en el sistema. Cada contenedor cumple una función específica dentro de la arquitectura, permitiendo desacoplar responsabilidades y facilitar la escalabilidad del sistema.

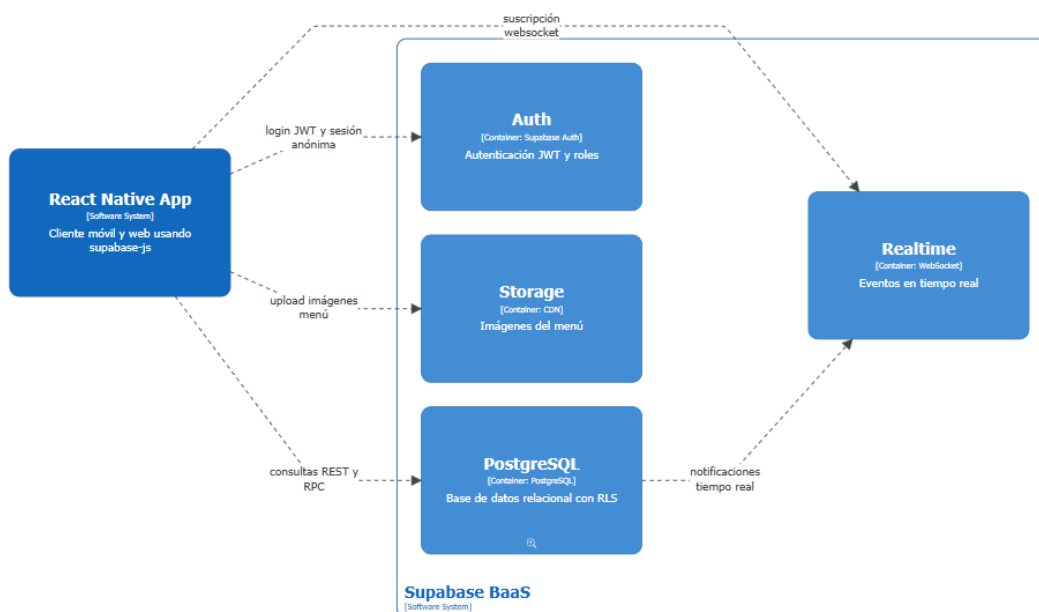


Figura 4.6. Contenedores principales del entorno Supabase

El contenedor PostgreSQL almacena la información relacionada con usuarios, mesas, pedidos, gastos y reportes. El servicio Auth permite gestionar sesiones de usuario mediante tokens JWT, soportando autenticación tradicional y sesiones anónimas utilizadas por la carta digital.

El servicio Storage permite almacenar imágenes de los productos del menú, las cuales son accesibles mediante URLs públicas optimizadas mediante CDN. Por otro lado, el servicio Realtime permite mantener sincronizados los estados de pedidos y llamadas de clientes mediante WebSockets.

4.3.5 Componentes de la Base de Datos

El siguiente diagrama describe los componentes internos de la base de datos implementada en Supabase. Se incluyen las tablas principales, sus relaciones lógicas y las funciones SQL utilizadas para generar reportes financieros y análisis de ventas.

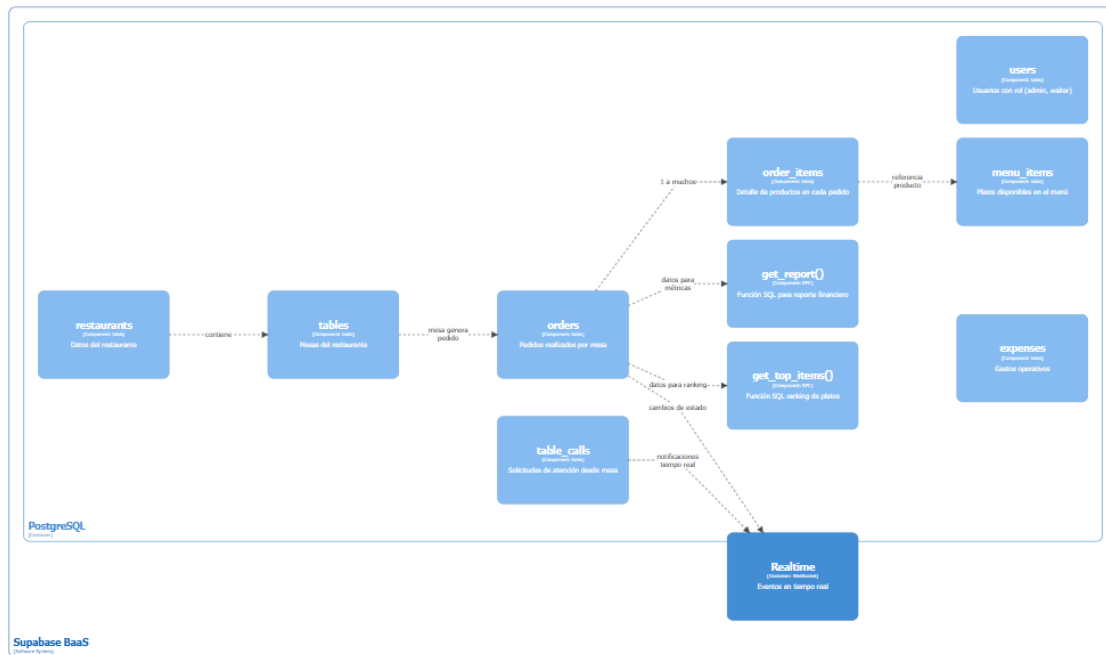


Figura 4.7. Componentes internos del esquema de base de datos en Supabase

La tabla `users` almacena los usuarios del sistema con sus respectivos roles. La tabla `restaurants` permite gestionar la información del restaurante, mientras que la tabla `tables` representa las mesas disponibles en el establecimiento.

Las tablas `orders` y `order_items` almacenan la información de los pedidos realizados, manteniendo consistencia histórica de precios mediante snapshots de los productos seleccionados. La tabla `expenses` permite registrar los gastos operativos necesarios para el cálculo de métricas financieras.

4.3.6 Vista de Estado de Mesas (Mesero)

La pantalla principal del mesero muestra en tiempo real el estado de cada mesa asignada. Esta funcionalidad se basa en la suscripción al canal `orders` de Supabase Realtime, permitiendo actualizar los estados de los pedidos sin necesidad de recargar la aplicación.

Adicionalmente, la aplicación escucha el canal `table_calls`, el cual permite detectar cuando un cliente solicita atención desde su mesa mediante la carta digital.

Estado visual	Condición en DB
Libre	Sin orden activa para esa mesa en la fecha actual
Pendiente	<code>status = 'pending'</code>
Listo para entregar	<code>status = 'ready'</code>
Entregado	<code>status = 'delivered'</code>
Cliente llama	Registro en <code>table_calls</code> con <code>attended = false</code>

Este mecanismo permite mantener sincronizados los estados entre múltiples dispositivos conectados simultáneamente, asegurando consistencia en la información mostrada al personal del restaurante.

5. Metodología Aplicada

5.1 Metodología de Desarrollo

Para el desarrollo del proyecto se adoptó la metodología ágil **Scrum**, dado que permite una alta flexibilidad y adaptación a los cambios, además de fomentar una comunicación constante entre los miembros del equipo. Scrum se basa en ciclos de trabajo cortos y entregables funcionales (sprints) que permiten un seguimiento continuo del progreso.

El proceso de desarrollo sigue las siguientes fases dentro del marco de Scrum:

- **Product Backlog:** Lista priorizada de los requisitos del proyecto, mantenida y priorizada por el Product Owner.
- **Sprint Planning:** Antes de cada sprint el equipo selecciona los ítems del backlog a completar y se compromete a entregarlos.
- **Daily Stand-Up:** Reuniones diarias breves para revisar progreso, detectar bloqueos y ajustar tareas.
- **Sprint Review:** Al final del sprint se presenta el incremento al Product Owner y partes interesadas.
- **Sprint Retrospective:** El equipo identifica áreas de mejora en el proceso y aplica ajustes al siguiente sprint.

5.2 Ciclo de Vida del Proyecto

El proyecto se organizó en **3 sprints de 4 semanas** cada uno:

Sprint	Alcance
1	Configuración del proyecto, diseño del schema en Supabase, autenticación, RLS, CRUD del menú e integración con Storage para manejo de imágenes.
2	Registro de pedidos, vista de mesas en tiempo real mediante Real-time, manejo de estados de pedido, módulo de gastos, funciones SQL de reportes y ranking de platos.
3	Carta digital pública por QR, sesión anónima por mesa, llamada al mesero, confirmación de pagos por admin, pruebas finales y despliegue con EAS Build.

Cuadro 5.1. *Release Plan — Sprints del proyecto*

5.3 Herramientas de Gestión

- **Notion:** Gestión del sprint backlog con vista Kanban (Por hacer / En progreso / En revisión / Completado), centralización de documentación técnica y seguimiento de tareas.

- **GitHub:** Control de versiones distribuido, revisión de código mediante pull requests y entrega de incrementos de forma controlada.
- **Discord:** Canal principal de comunicación cotidiana, dividido en espacios temáticos (desarrollo, diseño, pruebas).
- **Excalidraw:** Diagramación colaborativa de esquemas de base de datos, flujos de navegación y arquitectura del sistema.
- **Postman:** Testing y documentación de llamadas RPC y REST durante el desarrollo e integración.

6. Seguimiento y Desarrollo Grupal

6.1 Planificación del Trabajo

Sprint 1 — Fundación: base de datos, autenticación y roles

Objetivo: Setup del proyecto, schema de BD, autenticación, RLS y CRUD del menú.

Historias de Usuario

■ Setup e infraestructura

- HU-01: Inicializar proyecto Expo con TypeScript y Expo Router
- HU-02: Configurar proyecto Supabase y variables de entorno
- HU-03: Crear migración de esquema inicial de la base de datos
- HU-04: Crear constraint de orden activa única por mesa
- HU-05: Crear trigger set_closed_at al entregar una orden

■ Autenticación y roles

- HU-06: Implementar pantalla de login para mesero y admin
- HU-07: Proteger rutas según rol con guards de navegación
- HU-08: Crear trigger sync_role_to_auth para sincronizar rol
- HU-09: Implementar cierre de sesión

■ Row Level Security

- HU-10: Implementar RLS para la tabla orders
- HU-11: Implementar RLS para expenses, menu_items y table_calls

■ Gestión de usuarios (admin)

- HU-12: Listar usuarios del sistema (admin)
- HU-13: Crear nuevo usuario mesero o admin (admin)
- HU-14: Activar y desactivar usuario (admin)

■ Gestión de mesas (admin)

- HU-15: CRUD de mesas del restaurante (admin)
- HU-16: Gestionar restaurante y su slug (admin)

Sprint 2 — Operación: menú, pedidos, pagos y gastos

Objetivo: Registro de pedidos, vista de mesas en tiempo real y módulo financiero.

Historias de Usuario

■ Gestión de menú (admin)

- HU-17: Listar ítems del menú con filtro por categoría (admin)
- HU-18: Crear ítem del menú con imagen (admin)
- HU-19: Editar precio y descripción de un ítem (admin)
- HU-20: Activar o desactivar disponibilidad de un ítem (admin)
- HU-21: Eliminar ítem del menú (admin)

■ Toma de pedidos (mesero)

- HU-22: Ver el menú disponible al tomar un pedido (mesero)
- HU-23: Crear una nueva orden seleccionando mesa e ítems (mesero)
- HU-24: Ver detalle de una orden activa (mesero)
- HU-25: Agregar ítems a una orden existente (mesero)
- HU-26: Marcar una orden como entregada (mesero)
- HU-27: Cancelar una orden (mesero y admin)
- HU-28: Seleccionar método de pago al crear la orden (mesero)

■ Pagos (admin)

- HU-29: Ver órdenes con pago QR pendiente de confirmación (admin)
- HU-30: Confirmar pago QR de una orden (admin)
- HU-31: Mostrar QR estático de pago al cliente (mesero)

■ Gastos operativos (admin)

- HU-32: Registrar un gasto operativo (admin)
- HU-33: Listar gastos por rango de fechas (admin)
- HU-34: Editar y eliminar un gasto (admin)

Sprint 3 — Tiempo real, carta digital y reportes

Objetivo: Carta digital pública, sesión de cliente en mesa, pagos y despliegue final.

Historias de Usuario

■ Vista de mesas en tiempo real (mesero)

- HU-35: Ver estado de todas las mesas asignadas (mesero)
- HU-36: Actualizar estado de mesas en tiempo real sin recargar (mesero)

- HU-37: Recibir notificación cuando un pedido está listo (mesero)
- HU-38: Manejar reconexión automática del canal Realtime (mesero)
- **Llamada al mesero (cliente → mesero)**
 - HU-39: Mostrar botón "Llamar al mesero.^{en} la sesión de cliente en mesa
 - HU-40: Recibir notificación de llamada de cliente en tiempo real (mesero)
 - HU-41: Marcar llamada de cliente como atendida (mesero)
- **Carta digital pública (web)**
 - HU-42: Publicar carta digital accesible por URL sin login
 - HU-43: Mostrar carta organizada por categorías con imágenes y precios
 - HU-44: Generar y mostrar QR del restaurante para carta pública (admin)
- **Sesión de cliente en mesa (web)**
 - HU-45: Iniciar sesión anónima al escanear el QR de la mesa
 - HU-46: Ver menú completo (incluyendo no disponibles) desde la sesión de mesa
 - HU-47: Generar QR individual por mesa (admin)
- **Reportes financieros (admin)**
 - HU-48: Crear función SQL get_report para ingresos y egresos
 - HU-49: Crear función SQL get_top_items para ranking de platos
 - HU-50: Pantalla de reporte financiero por período (admin)
 - HU-51: Pantalla de ranking de platos más vendidos (admin)

51 historias de usuario · 3 sprints · 6 semanas

6.2 Roles y Responsabilidades

Integrante	Rol
Acebey Zalazar Jimmy Randall	Desarrollador
Cáceres Véliz Erick Alvaro	Desarrollador
Revollo Bernal Amilcar Diego	Scrum Master
Siles Espicho Adrian Eloy	Desarrollador
Tinta Mamani Jairo Amado	Desarrollador

6.3 Control de Versiones

El proyecto utiliza un sistema de control de versiones distribuido con **Git**, gestionado en un único repositorio alojado en GitHub (**gestion-comandas**). Este enfoque centralizado permite una coordinación eficiente entre el desarrollo del frontend móvil, las rutas web públicas y las migraciones de base de datos.

La estrategia de ramas adoptada es:

- **main**: rama de producción; solo recibe merges mediante Pull Requests aprobados.
- **feature/[nombre]**: una rama por historia de usuario o tarea de sprint.

6.4 Reporte de Avances

6.4.1 Indicadores de Progreso

ID	Historia de Usuario	Completada
HU-01	Inicializar proyecto Expo con TypeScript y Expo Router	✓
HU-02	Configurar proyecto Supabase y variables de entorno	✓
HU-03	Crear migración de esquema inicial de la base de datos	✓
HU-04	Crear constraint de orden activa única por mesa	✓
HU-05	Crear trigger set_closed_at al entregar una orden	✓
HU-06	Implementar pantalla de login para mesero y admin	✓
HU-07	Proteger rutas según rol con guards de navegación	✓
HU-08	Crear trigger sync_role_to_auth para sincronizar rol	✓
HU-09	Implementar cierre de sesión	✓
HU-10	Implementar RLS para la tabla orders	✓
HU-11	Implementar RLS para expenses, menu_items y table_calls	✓
HU-12	Listar usuarios del sistema (admin)	✓
HU-13	Crear nuevo usuario mesero o admin (admin)	✓
HU-14	Activar y desactivar usuario (admin)	✓
HU-15	CRUD de mesas del restaurante (admin)	✓
HU-16	Gestionar restaurante y su slug (admin)	✓
HU-17	Listar ítems del menú con filtro por categoría (admin)	✓
HU-18	Crear ítem del menú con imagen (admin)	✓
HU-19	Editar precio y descripción de un ítem (admin)	✓
HU-20	Activar o desactivar disponibilidad de un ítem (admin)	✓
HU-21	Eliminar ítem del menú (admin)	✓
HU-22	Ver el menú disponible al tomar un pedido (mesero)	✓

ID	Historia de Usuario	Completada
HU-23	Crear una nueva orden seleccionando mesa e ítems (mesero)	✓
HU-24	Ver detalle de una orden activa (mesero)	✓
HU-25	Agregar ítems a una orden existente (mesero)	✓
HU-26	Marcar una orden como entregada (mesero)	✓
HU-27	Cancelar una orden (mesero y admin)	✓
HU-28	Seleccionar método de pago al crear la orden (mesero)	✓
HU-29	Ver órdenes con pago QR pendiente de confirmación (admin)	✓
HU-30	Confirmar pago QR de una orden (admin)	✓
HU-31	Mostrar QR estático de pago al cliente (mesero)	✓
HU-32	Registrar un gasto operativo (admin)	✓
HU-33	Listar gastos por rango de fechas (admin)	✓
HU-34	Editar y eliminar un gasto (admin)	✓
HU-35	Ver estado de todas las mesas asignadas (mesero)	✓
HU-36	Actualizar estado de mesas en tiempo real sin recargar (mesero)	✓
HU-37	Recibir notificación cuando un pedido está listo (mesero)	✓
HU-38	Manejar reconexión automática del canal Realtime (mesero)	✓
HU-39	Mostrar botón "Llamar al mesero. ^{en} la sesión de cliente en mesa	✓
HU-40	Recibir notificación de llamada de cliente en tiempo real (mesero)	✓
HU-41	Marcar llamada de cliente como atendida (mesero)	✓
HU-42	Publicar carta digital accesible por URL sin login	✓
HU-43	Mostrar carta organizada por categorías con imágenes y precios	✓
HU-44	Generar y mostrar QR del restaurante para carta pública (admin)	✓
HU-45	Iniciar sesión anónima al escanear el QR de la mesa	✓
HU-46	Ver menú completo (incluyendo no disponibles) desde la sesión de mesa	✓
HU-47	Generar QR individual por mesa (admin)	✓
HU-48	Crear función SQL get_report para ingresos y egresos	✓
HU-49	Crear función SQL get_top_items para ranking de platos	✓
HU-50	Pantalla de reporte financiero por período (admin)	✓
HU-51	Pantalla de ranking de platos más vendidos (admin)	✓

Cuadro 6.1. Indicadores de progreso por historia de usuario

6.4.2 Dificultades Encontradas

- **Pausa automática del free tier de Supabase:** La base de datos se pausa tras períodos de inactividad, por lo que se implementó un ping periódico para mantener activo el servicio durante el desarrollo.
- **Reconexión de canales Realtime:** Las conexiones móviles inestables pueden interrumpir la sincronización en tiempo real. Se implementó manejo de eventos `CHANNEL_ERROR` y `TIMED_OUT` con reintento automático.
- **Persistencia de precios históricos:** Para mantener la consistencia financiera, el precio del producto se almacena como snapshot en el momento del pedido, evitando inconsistencias cuando el admin modifica el menú.

6.5 Comunicación

El equipo empleó las siguientes herramientas de comunicación y colaboración:

- **Notion:** planificación de tareas, backlog y documentación técnica.
- **GitHub:** control de versiones y revisión de código mediante pull requests.
- **Discord:** canal principal de comunicación durante el desarrollo.
- **WhatsApp:** coordinación rápida para decisiones puntuales.

7. Pruebas y Resultados

7.1 Plan de Pruebas

Se efectuaron evaluaciones funcionales, de usabilidad y de integración con el propósito de verificar el cumplimiento de los requisitos, analizar la facilidad de uso y garantizar la estabilidad del sistema en condiciones reales de operación.

7.1.1 Tipos de Pruebas Aplicadas

- **Pruebas funcionales:** Verificación de que cada módulo responde conforme a los requisitos funcionales (RF-01 a RF-12).
- **Pruebas de navegación:** Análisis de la coherencia de los flujos de pantalla, menús y transiciones entre roles.
- **Pruebas de integración:** Validación de la comunicación entre el cliente React Native y los servicios de Supabase (REST, RPC, Realtime, Storage).
- **Pruebas Realtime:** Verificación de la latencia de notificaciones en el canal `orders` y `table_calls` bajo condiciones de red móvil.
- **Pruebas de seguridad / RLS:** Intentos de acceso cruzado entre roles para validar que las políticas RLS bloqueen correctamente.
- **Pruebas exploratorias:** Recorrido libre de la aplicación por evaluadores externos para detectar comportamientos inesperados.
- **Pruebas de UX/UI:** Evaluación de la experiencia del usuario, priorizando interfaces simples y claras.

7.1.2 Escenarios Evaluados

1. Registro y login de mesero y administrador.
2. Creación de un pedido con selección de platillos, mesa y método de pago.
3. Actualización del estado de un pedido (`pending` → `ready` → `delivered`) y verificación de notificación en tiempo real.
4. Vista de estado de mesas: validar que los cambios de estado aparecen sin recargar la pantalla.
5. Llamada al mesero desde la sesión de cliente en mesa y recepción de notificación en el dispositivo del mesero.
6. Acceso a la carta digital pública (sin login) y validación de que solo aparecen ítems con `available = true`.
7. Gestión del menú por el admin: crear, editar precio, cambiar disponibilidad, subir imagen.

8. Registro de gastos operativos y generación de reporte financiero por rango de fechas.
9. Consulta del ranking de platos más vendidos (RPC `get_top_items`).
10. Confirmación de pago QR por el admin y verificación de contabilización en el reporte.
11. Intento de acceso cruzado: mesero intentando acceder a reportes (debe ser bloqueado por RLS).
12. Simulación de pérdida de conexión y reconexión automática del canal Realtime.

7.1.3 Herramientas de Soporte

- **Emuladores Android (Android Studio):** Pruebas preliminares de compatibilidad y funcionalidad básica.
- **Dispositivos físicos Android:** Validaciones de rendimiento y experiencia real del usuario final.
- **Expo Go / EAS Build:** Distribución de builds de prueba al equipo.
- **Supabase Dashboard:** Inspección directa de la base de datos y logs de RLS para verificar políticas.
- **Postman:** Pruebas directas de funciones RPC y endpoints REST.

7.1.4 Condiciones de Aprobación

Una prueba se considera superada cuando:

- Las funcionalidades operan conforme a los requisitos funcionales y criterios de aceptación definidos.
- Las políticas RLS bloquean accesos no autorizados sin excepción.
- Las notificaciones Realtime llegan en menos de 2 segundos en red WiFi.
- La experiencia del usuario es valorada como satisfactoria.
- No se detectan fallos críticos en las pruebas exploratorias.

7.2 Resultados Obtenidos

7.2.1 Registro y Gestión de Pedidos

El flujo de creación de pedidos fue validado exitosamente en dispositivos físicos. El mesero puede seleccionar platillos del menú, asignar una mesa y elegir el método de pago. El cálculo del total se realiza automáticamente al agregar ítems.

7.2.2 Vista de Mesas en Tiempo Real

La pantalla de estado de mesas del mesero actualizó correctamente los estados (**pending**, **ready**, **delivered**) al recibir eventos del canal **orders** de Supabase Realtime, sin necesidad de recargar la pantalla. La latencia promedio observada fue inferior a 1.5 segundos en red WiFi.

7.2.3 Carta Digital y Sesión de Cliente

La carta digital pública fue accedida correctamente desde navegadores móviles sin descarga de app. La sesión anónima por mesa se inició correctamente al escanear el QR de mesa y el cliente pudo llamar al mesero, recibiendo el mesero la notificación en tiempo real vía canal **table_calls**.

7.2.4 Reportes Financieros

Las funciones RPC **get_report** y **get_top_items** devolvieron resultados correctos en todas las combinaciones de rango de fechas probadas. La validación de rol dentro de la función **get_report** bloqueó correctamente el acceso cuando se intentó invocar con un token de rol **waiter**.

7.2.5 Seguridad RLS

Todos los intentos de acceso cruzado entre roles fueron bloqueados por las políticas RLS. El mesero no pudo leer órdenes de otros meseros ni acceder a la tabla **expenses**. El cliente anónimo solo pudo insertar en **table_calls** para su propia mesa.

7.3 Limitaciones Conocidas

7.3.1 Free Tier de Supabase

Límite	Valor
Base de datos	500 MB
Storage	1 GB
Requests mensuales	50,000
Conexiones Realtime simultáneas	200
Inactividad (pausa automática)	7 días sin actividad

Advertencia

La pausa automática por inactividad es un riesgo operacional real en producción. Debe resolverse mediante un ping periódico (ej. **cron-job.org**) o migración al tier Pro (\$25/mes) antes del primer despliegue en un restaurante real.

7.3.2 Limitaciones de Alcance Documentadas

- La **reasignación de órdenes** entre meseros no está soportada en tiempo real. Si el admin reasigna una orden, el nuevo mesero no recibe el evento Realtime hasta el próximo ciclo de polling.

- El **cliente en mesa no puede crear órdenes directamente**. Requeriría un estado **draft** y un flujo de confirmación por el mesero, fuera del alcance actual.
- El **pago QR es estático** y su verificación es manual por el admin. No hay integración con pasarelas de pago en este alcance.
- Las **cancelaciones parciales y devoluciones** no tienen flujo dedicado. Se recomienda implementar en versiones posteriores.

7.4 Mejoras Propuestas

1. **Pasarela de pagos:** Integrar con una pasarela (ej. Mercado Pago o Stripe) para verificar pagos QR de forma automática.
2. **Dashboard live global:** Implementar una vista de operación en tiempo real para el admin con estado de todas las mesas y meseros (descartada del alcance actual por complejidad).
3. **Cancelaciones parciales:** Añadir flujo para devolver o cancelar ítems individuales de una orden sin cancelar toda la comanda.
4. **Reasignación de órdenes en tiempo real:** Soporte completo de Realtime al reasignar una orden a otro mesero.
5. **Soporte multi-idioma:** Internacionalización de la interfaz para restaurantes con clientes internacionales.
6. **Modo offline básico:** Caché local de las órdenes activas para operar sin conexión y sincronizar al reconectar.

8. Conclusiones

El desarrollo del proyecto **Gestión de Comandas** representa un esfuerzo exitoso que logró cumplir con los objetivos planteados, consolidando una plataforma integral para la modernización operativa de restaurantes pequeños y medianos. La aplicación implementa funcionalidades clave como el registro de pedidos en tiempo real, la vista de estado de mesas vía Supabase Realtime, la gestión diferenciada por roles con Row Level Security, los reportes financieros calculados server-side mediante funciones PostgreSQL, y la carta digital pública accesible sin app desde cualquier dispositivo.

Este logro fue posible gracias a la adopción del modelo BaaS con Supabase, que permitió eliminar la necesidad de un backend intermedio propio, reduciendo la complejidad operacional y el costo de infraestructura. La elección de React Native con Expo garantiza actualizaciones OTA sin pasar por las tiendas de aplicaciones, lo que es especialmente valioso en un entorno de restaurante donde la continuidad del servicio es crítica.

El trabajo en equipo bajo la metodología **Scrum** demostró ser fundamental para el progreso organizado del proyecto, permitiendo una distribución efectiva de responsabilidades y una supervisión constante del avance a través de sprints bien definidos. La asignación clara de roles, combinada con el uso estratégico de herramientas colaborativas como GitHub, Notion y Discord, facilitó la resolución de desafíos técnicos y aseguró la entrega de incrementos funcionales en cada iteración.

Si bien la plataforma alcanzó sus objetivos principales, algunas funcionalidades avanzadas, como la integración con pasarelas de pago, el dashboard live global para el administrador y las cancelaciones parciales, representan oportunidades de mejora para futuras iteraciones que podrán construirse sobre la base tecnológica y funcional sólida establecida en este desarrollo.

8.1 Recomendaciones

8.1.1 Futuros Desarrollos

- **Integración con pasarelas de pago:** Conectar con Mercado Pago, Stripe u otra pasarela disponible en el mercado local para automatizar la verificación de pagos QR y eliminar la confirmación manual.
- **Dashboard live global:** Implementar la vista global de operación del restaurante para el admin, mostrando el estado de todas las mesas y meseros en tiempo real.
- **Cancelaciones parciales y devoluciones:** Añadir un flujo dedicado para devolver o cancelar ítems individuales dentro de una orden.
- **Modo offline básico:** Implementar caché local de órdenes activas para operar sin conexión y sincronizar al reconectar, usando tecnologías como MMKV o SQLite en el cliente React Native.
- **Soporte multi-sucursal avanzado:** La tabla `restaurants` ya habilita multi-sucursal; la siguiente iteración puede añadir un panel de administración por cadena de restaurantes.

8.1.2 Optimizaciones Técnicas

- **Migración al tier Pro de Supabase:** Eliminar el riesgo de pausa automática por inactividad antes del despliegue en producción real.
- **Índices adicionales:** Añadir índices compuestos en `orders(waiter_id, status)` y `order_items(order_id, menu_item_id)` para optimizar las consultas más frecuentes en horas pico.
- **Pruebas automatizadas:** Incorporar pipelines de CI/CD con suites de testing unitario (Jest), integración y end-to-end (Detox) para reducir la deuda de regresión.
- **Monitoreo de rendimiento:** Integrar Sentry o similar para captura de errores y métricas de rendimiento en producción.

8.1.3 Gestión y Comunicación

- **Documentación técnica continua:** Establecer un proceso sistemático para mantener actualizados el esquema de BD, las políticas RLS y las decisiones de arquitectura en el repositorio.
- **Protocolos de code review:** Definir checklists estandarizados para revisiones de código que aseguren calidad y consistencia.
- **Capacitación en herramientas:** Realizar sesiones de entrenamiento en el uso avanzado de Supabase (migraciones, funciones RPC, Realtime) para todo el equipo de desarrollo.

9. Bibliografía

9.1 Tecnologías y Frameworks

- Supabase. (s/f). *Supabase Documentation*. Recuperado el 11 de noviembre de 2025, de <https://supabase.com/docs>
- Supabase. (s/f). *supabase-js v2 Reference*. Recuperado el 11 de noviembre de 2025, de <https://supabase.com/docs/reference/javascript>
- Supabase. (s/f). *Anonymous Sign-ins*. Recuperado el 11 de noviembre de 2025, de <https://supabase.com/docs/guides/auth/anonymous-sign-ins>
- Expo. (s/f). *Expo Documentation*. Recuperado el 11 de noviembre de 2025, de <https://docs.expo.dev/>
- Expo. (s/f). *Expo Router — Web Support*. Recuperado el 11 de noviembre de 2025, de <https://docs.expo.dev/router/introduction/>
- Expo. (s/f). *EAS Build*. Recuperado el 11 de noviembre de 2025, de <https://docs.expo.dev/build/introduction/>
- React Native. (s/f). *React Native Documentation*. Recuperado el 11 de noviembre de 2025, de <https://reactnative.dev/docs/getting-started>
- PostgreSQL Global Development Group. (s/f). *PostgreSQL Row Level Security*. Recuperado el 11 de noviembre de 2025, de <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>

9.2 Herramientas de Desarrollo

- GitHub Inc. (s/f). *GitHub Documentation*. Recuperado el 11 de noviembre de 2025, de <https://docs.github.com>
- Notion Labs. (s/f). *Notion Help & Support*. Recuperado el 11 de noviembre de 2025, de <https://www.notion.com/help>
- Excalidraw. (s/f). *Excalidraw — Collaborative Whiteboard*. Recuperado el 11 de noviembre de 2025, de <https://excalidraw.com/>
- Postman Inc. (s/f). *Postman Documentation*. Recuperado el 11 de noviembre de 2025, de <https://learning.postman.com/docs/>

9.3 Referencias Institucionales

- Universidad Mayor de San Simón. (s/f). *Ingeniería Informática — UMSS*. Recuperado el 11 de noviembre de 2025, de <https://www.umss.edu.bo/licenciatura-en-ingenieria-en>